Deep Learning Algorithms

Second Edition

In PyTorch

(GPTs to MLPs)

Deep Learning Algorithms

Second Edition

In PyTorch
(GPTs to MLPs)

Ricardo A. Calix

Copyright © 2025 by *Ricardo A. Calix*.

https://www.rcalix.com

For more information about this book series, write to rcalix@rcalix.com.

2025, Second Edition

ISBN 979-8-856475-78-3

Illustrations by: *Ricardo A. Calix*
All rights reserved. No part of this work may be reproduced or transmitted in any form or by any means, without written permission of the copyright owner.

Dedication

To my wife Valerie and my daughter Summer.

Acknowledgements

Thanks to my students and colleagues at Purdue University Northwest who helped me to develop a Big Data course which is the basis for this book. I especially want to thank Professor Keyuan Jiang for his support of my teaching and research endeavors. I want to thank Ravish Gupta and Matrika Gupta for their questions and comments that helped to improve the material discussed in this book. I would also like to thank my Fall 2018, 2019 deep learning students for their help in making this book a better instrument for learning. For improvements to this second edition, I want to thank my Spring and Fall 2023 ML/DL students. I especially want to thank students Joseph Shapiro, David Higley, and Danielle Turner for their amazing projects and great coding skills which have made it into some of the pages of this book. Finally, I want to thank all My previous AI teachers including my Ph.D. advisor Prof. Gerald Knapp, other university professors such as Prof. Bahadir Gunturk, as well as all the great YouTube lecturers out there such as Andrew Ng and Andrej Karpathy.

Contents

Dedication v

Acknowledgements vii

Preface xv

1 Introduction 1
- 1.1 Setting up your Environment . . . 2
  - 1.1.1 Anaconda . . . 2
  - 1.1.2 Setting up your Environment with a Physical Box . . . 2
- 1.2 Background . . . 3
- 1.3 Companion GitHub Code and YouTube videos . . . 3
- 1.4 Numpy Arrays, Tensors, and Linear Algebra . . . 4
  - 1.4.1 Numpy Arrays . . . 4
  - 1.4.2 Tensor Operations with PyTorch . . . 15
- 1.5 Conclusion . . . 19

2 Traditional Machine Learning 21
- 2.1 Code Issues . . . 22
- 2.2 Object Oriented Programming . . . 25
- 2.3 Performance Evaluation . . . 26
  - 2.3.1 Regression Performance Evaluation . . . 26
  - 2.3.2 Classification Performance Evaluation . . . 27
  - 2.3.3 Plotting Performance . . . 29
- 2.4 Optimization . . . 31
- 2.5 KNN . . . 34
- 2.6 Logistic Regression . . . 39
- 2.7 Neural Networks . . . 40
- 2.8 Support Vector Machines (SVM) . . . 41
- 2.9 Regression, Trees, and XGBoost . . . 43
  - 2.9.1 Regression Trees . . . 43
  - 2.9.2 XGBoost . . . 43
- 2.10 The t-test for Machine Learning . . . 43
- 2.11 Summary . . . 44

3 Data Loading and Pre-processing 45
- 3.1 Loading the Data . . . 45

3.2 Data and Feature Pre-Processing . . . 48
3.3 One Hot Encoding . . . 49
3.4 Features . . . 51
3.4.1 Features from Text . . . 52
3.4.2 Features from Images . . . 53
3.5 Corpora (i.e. Annotated Datasets) . . . 58
3.6 Summary . . . 60

4 Deep Learning: Starting at the beginning 61
4.1 Things to know about the code . . . 61
4.2 Getting Started with Deep Leaning . . . 64
4.3 Deep Leaning Definition . . . 67
4.4 PyTorch Basics . . . 67
4.5 Loading your Data into your PyTorch code . . . 69
4.6 Linear Regression . . . 73
4.6.1 Theory and Intuition of Linear Regression . . . 73
4.6.2 Linear Regression model for Wine Quality data . . . 77
4.7 Linear Regression to Logistic Regression and Neural Networks . . . 93
4.7.1 Regression vs Classification . . . 93
4.7.2 Instantiating the different models . . . 94
4.7.3 Reading Data in batches . . . 95
4.7.4 What is Number of Epochs? . . . 96
4.8 Logistic Regression . . . 97
4.8.1 Theory and Intuition of Logistic Regression . . . 97
4.8.2 Entropy and Cross Entropy . . . 104
4.8.3 Logistic Regression NN Architecture . . . 117
4.9 Layers of the Neural Network in PyTorch . . . 119
4.9.1 Intuition of adding more layers to a Neural Network . . . 121
4.10 Going Deep: An N layer Neural Network in PyTorch . . . 121
4.11 Easy Deep Learning with the Iris dataset . . . 124
4.12 More Challenging Deep Learning with the Wine Quality dataset . . . 138
4.13 Summary . . . 150

5 Semantic Vector Spaces 151
5.1 Vector Space Model . . . 151
5.2 Latent Semantic Analysis (LSA) . . . 153
5.3 Word Embeddings and Neural Networks . . . 155
5.4 Word2Vec code . . . 160
5.5 Knowledge Enhanced Vector Spaces . . . 176

5.6 Deep Gramulator . . . 178
5.7 Summary . . . 179

6 Convolutional Neural Networks 181
6.1 The Data . . . 181
6.2 Convolution and CNNs Defined . . . 181
6.3 Architecture for a Convolutional Neural Network with MNIST . . . 182
6.4 Coding a CNN for MNIST . . . 185
6.4.1 Figuring out the tensor shapes after the convolution layers . . . 193
6.5 Summary . . . 194

7 Recurrent Neural Networks 195
7.1 Using an RNN to classify MNIST . . . 195
7.1.1 Reshaping Tensors for use with RNNs and dummy data . . . 197
7.1.2 Classifying MNIST with RNNs . . . 199
7.2 Transformers . . . 209
7.3 Summary . . . 209

8 Generative Adversarial Networks 211
8.1 Autoencoders . . . 212
8.2 Original GAN . . . 212
8.2.1 Generating MNIST digits with GANs . . . 212
8.3 Conditional GANs . . . 224
8.4 Summary . . . 225

9 Reinforcement Learning 227
9.1 So what is Q-Learning? . . . 227
9.2 Q-Learning using a Table . . . 230
9.3 Q-Learning using a Neural Network . . . 235
9.4 Performance, Q-NN, and randomness . . . 248
9.5 Deep Deterministic Policy Gradient . . . 253
9.6 SAC . . . 254
9.7 Proximal Policy Optimization . . . 255
9.8 Reinforcement Learning with Human Feedback . . . 256
9.9 Direct preference optimization . . . 256
9.10 Summary . . . 258

10 Transformers **259**
10.1 Main Ideas of the Transformer . . . 260
10.1.1 Numpy arrays, tensors, and linear algebra . . . 261
10.1.2 Attention . . . 261
10.1.3 Embeddings . . . 262
10.1.4 Masks . . . 263
10.1.5 Positional Encoding . . . 263
10.1.6 Tokens . . . 263
10.1.7 Inputs and Outputs . . . 264
10.2 GPTs . . . 265
10.2.1 Teacher forcing . . . 267
10.2.2 Implementing a GPT in PyTorch from Scratch . . . 267
10.2.3 Prompting . . . 292
10.2.4 Linear Algebra operations to understand Attention . . . 294
10.3 Encoder Only Models . . . 302
10.3.1 BERT . . . 304
10.4 Encoder Decoder Transformers . . . 307
10.5 Data Wrangling from Scratch . . . 309
10.6 Summary . . . 316

11 Transfer Learning **317**
11.0.1 Important Ideas in Transfer Learning . . . 317
11.0.2 Pre-training . . . 317
11.0.3 Fine-tuning . . . 317
11.0.4 Freezing some weights of the neural network . . . 318
11.0.5 Sharing the weights from one model to another . . . 320
11.0.6 Reinforcement Learning Through Human Feedbacks (RLHF) . . . 320
11.0.7 Transfer Learning Performance Metrics . . . 320
11.1 Summary . . . 321

12 Conclusions and Final Thoughts **323**
12.1 The AI horizon . . . 323
12.2 Summary . . . 324

Other Books by Ricardo A. Calix **325**

The Author: Ricardo A. Calix **327**

Glossary **329**

Bibliography **331**

An Important Final Note **333**

Preface

“ The early bird catches the worm.

This is the second edition of my book “Deep Learning Algorithms”. A lot has changed since my original book. I wrote the first book in TensorFlow 1. After that, Google did a complete re-write of TensorFlow to create, at least, two ecosystems which include: low level TensorFlow 2 and high level Keras. I did not like either one and the learning curve was so significant that I just decided to switch to PyTorch. I am very happy with my decision. I have enjoyed learning PyTorch. I write these books basically to explain these topics to myself in the future in a way that is easy for me to understand. I make them available on Amazon for others to use if they find them useful. It is now 2023 and Deep Learning has taken off like a rocket. This year some people believed we had reached the singularity. It is definitely an exciting time. Deep Learning techniques are widely used by companies now and the algorithms are doing things that are truly amazing. Generative models are now very popular and whole ecosystems and industries are gravitating towards them. This book will still cover the basics and focuses on building things from scratch. The term ”From Scratch” today means writing code in PyTorch. Therefore, the goal of this book is to help you and me to learn, and to better understand, how to write deep learning algorithms from scratch using PyTorch. As is necessary with progress, the algorithms are now more complicated, with deeper and more resource intensive neural networks. This is best exemplified by one of the newest deep learning algorithms: The Transformer. Transformers are, for me, the first algorithm I was not able to train on a laptop. They truly require lots of GPU power and memory. The algorithms are much more complicated too, a little bit too much. In fact, the programming languages are starting to abstract too much of the code. I have found that a simple solution to this problem is to write everything in an object oriented fashion. This way, you are still defining the classes from scratch but you can also build far more complicated algorithms thanks to traditional object oriented programming techniques. A GPT can easily be defined using a reasonable amount of lines of code if leveraging object oriented techniques.

Ricardo A. Calix, Ph.D.

Greater Chicago Area, USA, December 20, 2023



1 Introduction

This book is very sequential and I recommend that you start from chapter 1 and read it sequentially, especially if you are new to deep learning. If you are experienced in deep learning, then you can probably skip around the chapters. In this book I will discuss some aspects of the theory of deep learning while providing as much intuition as possible. I will use Linux or mac or windows, python, Sklearn, and PyTorch. Chapter 1 will briefly discuss some of the background required or recommended for deep learning. In chapter 2, I will address some of the traditional machine learning topics. This will be a brief review of some of the traditional (non-deep learning based) machine learning (ML) algorithms for context. I will introduce the SKLearn library to provide code examples on how to quickly use the different ML models, and other tools. I will quickly move through regression to arrive at other popular algorithms. There are many books on the theory of these traditional machine learning algorithms (e.g. Witten and Frank, etc.). So, instead of discussing the theory, I will concentrate on the practical aspects of using these traditional machine learning algorithms in Python. More importantly, I will show how Sklearn can later be used with PyTorch for deep learning. Finally, in chapter 2, I will discuss evaluation tasks and performance metrics. In chapter 3, I will address the very important issue of the data and data pre-processing. To me and most practitioners, data collection and data processing present some of the most important issues, and sometimes difficult problems, in machine learning/deep learning. There are many issues that must be addressed when dealing with data. These include: getting the data, cleaning data, pre-processing data, building a corpus and annotating it, performing inter-annotator agreement analysis, etc. Some of these issues will be discussed in chapter 3. In chapter 4, I will introduce the topic of deep learning for the first time and, in particular, how to create deep neural networks. My goal here is to help students and practitioners alike to better understand the modeling pipeline. In particular, I want the reader to be able to get data and pre-process it in the appropriate format for deep learning. This chapter also covers how to load data to your program, and how to perform simple modeling tasks with plain vanilla deep neural networks of 2 or more hidden layers.

Chapter 5 covers problems related to vector spaces. Chapter 6 introduces the popular CNN methodology for image analysis. Chapter 7 coves RNNs and how neural networks can help with sequential data. Chapter 8 discusses GANs which are very exciting image based generative models. Chapter 9 cover Reinforcement Learning (RL) and some of its exciting ideas.

In the last chapters I will introduce you to the very broad topic of Transformers and the mechanism of Attention. This is one of the latest developments in deep learning as of 2017.

Transformers are very powerful and offer a lot of promise for NLP. Finally, in the final chapters I will discuss some final loose ends as well as present my final thoughts and conclusions. My goal for this book is to present intuition over equations and to use code to convey how things work. This is how I like to learn. To quote Richard Feynman: "What I cannot create, I do not understand". Some equations can still be useful, however, so I will use them when appropriate. Additionally, all the code used in this book can be obtained from GitHub at https://github.com/rcalix1 and any other complimentary materials about the book such as some of the figures in color can be obtained from the book website at www.rcalix.com.

1.1 Setting up your Environment

In this section I will discuss how to set up your environment to get started with deep learning programming. The 2 main environments I will discuss are using Anaconda, and building the hardware with a CPU and GPU.

1.1.1 Anaconda

I recommend you use Anaconda. It can be installed on Linux, windows, or mac and works exceptionally well. Go to www.anaconda.org and download the software from there. You will then create your anaconda environment and install most of your libraries using "pip".

You can get more detailed instructions to install PyTorch from (www.PyTorch.org) and for Sklearn from (http://scikit-learn.org/).

To get the most out of deep learning by using deep architectures and massive amounts of data you will need to use a GPU enabled machine. I suggest using the cloud or building your own GPU box. The cloud is probably the way to go as of 2024.

1.1.2 Setting up your Environment with a Physical Box

In this section I will give you some of the specifications of a machine I have used for the code in this book. In general, the more powerful the machine the better it will be at processing large amounts of data. Here are the specifications I have used:

- A GPU GeForce RTX2080 Ti or better (H100)
- A CPU such as the AMD 12 CORE
- Power supply EVGA SuperNOVA 1600 W P2 220

- Motherboard for multiple GPUs and CPUs
- More than 64 GB of RAM (DDR4)
- SSD hard drive 2 TB
- A case with cooling

The total cost for 1 device with just 1 CPU and 1 NVIDIA GPU may be between $ 4,500 and $ 5,500. Frankly, I think it is no longer feasible to build your GPU at these prices. So looking into the cloud is important. But, as of today, I can still use my PC with one GPU (in 2023).

1.2 Background

So, what background should you have to get the best out of deep learning? This is a question that often comes up and it is very important. Depending on what you will do with deep learning, I would recommend a course in statistics that covers probability and linear regression, programming up to data structures with Python and C/C++, a course in optimization, and a course on linear algebra. Sometimes a course in computer graphics using something like openGL where you have to manipulate meshes via linear algebra operations can also be very helpful to visualize and better understand matrices and vectors. Of all of these, matrix and vector operations from linear algebra are absolutely essential for writing algorithms from scratch. The name Tensor in PyTorch means matrix of any dimension. So, even the name reminds you of the importance of linear algebra for deep learning. While this is not a linear algebra book, I will provide in the next sections a quick introduction to the topic and some useful code examples that might help you later as you progress through the different deep learning algorithms. For a better treatment of linear algebra and optimization I recommend "Linear Algebra and Optimization for Machine Learning: A Textbook" by Charu C. Aggarwal or Gilbert Strang's books.

1.3 Companion GitHub Code and YouTube videos

All companion GitHub Code and YouTube videos can be found here:

https://github.com/rcalix1/DeepLearningAlgorithms

or at

https://www.rcalix.com

1.4 Numpy Arrays, Tensors, and Linear Algebra

In this section, I will present some of the most useful tensor operations and techniques used in this book, or in the deep learning field as a whole. In general, we want to be very efficient in our processing of the data so we do not use python lists and "for" loops when possible. Instead, we use Numpy arrays and tensors. These are treated as vectors and matrices in linear algebra. And more generally are referred to as tensors. The advantage of this approach is that we can perform a lot of linear algebra based math operations like the matrix multiplication, the transpose, etc. on our data using Python's Numpy or PyTorch libraries. The best way to learn about this approach is to do a series of exercises. I will start with examples of Numpy array operations and then move to tensor operations with PyTorch. Along the way I will point out some terminology or concepts from linear algebra.

1.4.1 Numpy Arrays

Okay, so let's get started. First we need to import the Numpy and PyTorch libraries.

Listing 1.1: Libraries

```
import numpy as np
import matplotlib.pyplot as plt
import torch
```

Once we import the libraries we can start writing some code. Let's begin with some warm up examples using Numpy.

To declare an array in Numpy we can write the following:

Listing 1.2: Declare an array

```
a = np.array([4,5,2,6,8])
print(a)
```

This results in

[4 5 2 6 8]

We can also declare a Numpy array with different data types. For instance, an array can be declared as float.

Listing 1.3: Declare an array as float

```
a = np.array([1, 3, 2, 5] , dtype='float32' )
print(a)
```

This gives us

[1. 3. 2. 5.]

A numpy matrix (2D np array) can be declared like so

Listing 1.4: Declare 2D array

```
list_of_lists = [[1, 2, 3],
                 [4, 4, 5] ,
                 [6, 2, 11]]

b = np.array(list_of_lists)
print(b)
```

This gives us the following 3x3 matrix.

Listing 1.5: 2d array result

```
[[ 1  2  3]
 [ 4  4  5]
 [ 6  2 11]]
```

Numpy has special functions to initialize a matrix. For example

Listing 1.6: Zeros matrix

```
b = np.zeros(10, dtype=int)
print(b)
```

The previous code generates a Numpy array of size 10 made up of all zeros.

[0 0 0 0 0 0 0 0 0 0]

We can also create a matrix of all ones with the following function:

Listing 1.7: All ones matrix code

```
b = np.ones((4, 6), dtype=float)
print(b)
```

Our previous code produces a 4x6 matrix of type ”float”.

Listing 1.8: All ones matrix

```
[[1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]]
```

For a matrix made up of just one single value we can write:

Listing 1.9: Matrix with one value

```
b = np.full((3, 3), 42)
print(b)
```

This generates a 3x3 matrix where all values are 42 as can be seen below.

Listing 1.10: result of matrix with one value

```
[[42 42 42]
 [42 42 42]
 [42 42 42]]
```

Sometimes we need Numpy arrays with different data in them. Numpy has quick ways of creating arrays with different data in them. The **np.arange** function is useful for this. For example:

Listing 1.11: numpy arange function

```
b = np.arange(1, 30, 3) print(b)
```

The previous code gives us a Numpy array with 10 values in the range from 1 to 28 with a step of size 3.

```
[ 1 4 7 10 13 16 19 22 25 28]
```

The function **linspace** is another way of generating numpy arrays with data in them. Here we generate 20 data points from 0 to 1 spaced by a step size of around 0.05.

Listing 1.12: The function linspace

```
b = np.linspace(0, 1, 20)

print(b)
```

The output looks like this:

Listing 1.13: The function linspace output

```
[ 0.         0.05263158 0.10526316 0.15789474 0.21052632 0.26315789
  0.31578947 0.36842105 0.42105263 0.47368421 0.52631579 0.57894737
  0.63157895 0.68421053 0.73684211 0.78947368 0.84210526 0.89473684
  0.94736842 1.        ]
```

Yet another useful function is **np.random.random**.

Listing 1.14: The function np.random.random

```
b = np.random.random((4, 4))

print(b)
```

With this function we can generate random data like the following. Here we have a 4x4 matrix with random values in it.

Listing 1.15: The function np.random.random output

```
[[0.52467069 0.68216617 0.79782109 0.33720887]
 [0.67956722 0.04082517 0.31311017 0.72985649]
 [0.64533659 0.83448976 0.37986602 0.60524177]
 [0.98868748 0.36999339 0.33000013 0.04157917]]
```

For random data with a mean of 0 and standard deviation of 1 we can write the following code:

Listing 1.16: Random data with a mean and standard deviation

```
## mean 0 and standard deviation 1
b = np.random.normal(0, 1, (4,4))
print(b)
```

Here the data looks like the following where we get a 4x4 matrix of random data with mean 0 and standard deviation 1.

Listing 1.17: Random data with distribution - output

```
[[ 0.82064949 -0.95219825 -1.27123377 -1.01187383]
 [ 0.44419588  0.17695603 -0.75775624 -0.14476445]
 [ 0.59233303  1.27530445  0.77260354 -0.80240966]
 [-0.58009786 -1.04106833 -1.27650071  0.28198804]]
```

Sometimes when doing linear algebra operations you need special matrices like the identity matrix. You can generate this matrix with numpy using the following code:

Listing 1.18: The identity matrix

```
## indentity matrix

b = np.eye(5)
print(b)
```

The previous code generates a 5x5 identity matrix like the following with one values only on the diagonal.

Listing 1.19: The identity matrix output

```
[[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
```

Okay. Now let's practice generating some matrices and looking at their dimensions.

Listing 1.20: Generate matrices

```
b1 = np.random.randint(20, size=6)
b2 = np.random.randint(20, size=(3,4))
b3 = np.random.randint(20, size=(2,4,6))

print(b2)
print(b3)
print("b2 dims ", b2.ndim)
print("b3 shape ", b3.shape)
print("b2 size ", b2.size)
print("data type of b3 ", b3.dtype)
```

The previous code generates several tensors with different dimensions:

We generate a matrix of size [3, 4] like the following:

Listing 1.21: Matrix of size (3, 4)

```
[[ 7 13 16 13]
 [11  6 17 11]
 [ 0 12  6  4]]
```

We also generate a matrix of size [2, 4, 6] as shown in the next listing.

Listing 1.22: Matrix of size (2, 4, 6)

```
[[[12  10  15   3  14   4]
  [ 9   7   4   0  16  10]
  [11  16   9   0   5  12]
  [ 4  12   6   9   3   5]]

 [[ 7  17   5  18   0  15]
  [ 9   3   4   4   7   0]
  [15   1   4  12  10  17]
  [ 9  14   1  14  19   8]]]
```

Knowing how to index a Numpy array is very important. Here we have an example of how to extract values by index.

Listing 1.23: Indexing

```
## indexing
a = np.array([1, 3, 2, 5] , dtype='float32' )

print(a)
print("first ", a[0])
print("third ", a[2])
print("last ", a[-1])
print("before last ", a[-2])
```

The results of the previous indexing examples can be seen in the next listing.

Listing 1.24: Indexing results

```
[1. 3. 2. 5.]
First value  1.0
Third value  2.0
Last value  5.0
before last  value 2.0
```

We can also do indexing on 2D matrices as follows:

Listing 1.25: Indexing on 2D matrices

```
## indexing
a = np.array([
               [1, 2, 3, 4],
               [5, 6, 7, 8],
               [9, 10, 11, 12]
             ] )

print(a)
print("first ", a[0,0])
print("last ", a[2, -1])
```

The previous indexing examples give us the following results for the 3x4 matrix

Listing 1.26: Indexing on 2D matrices - results

```
    [[ 1  2  3  4]
     [ 5  6  7  8]
     [ 9 10 11 12]]

## We get the first and last values

    first  1
    last  12
```



Figure 1.1: Like slicing a cake

One important concept when dealing with Numpy arrays or tensors is slicing. Slicing helps us to extract slices of data from a matrix like extracting 2 middle column vectors in a matrix. The following are some examples of slicing:

Listing 1.27: Examples of Slicing

```
## slicing
x = np.arange(15)
print(x)
print("first 4 elemets ", x[:4])
print("all after 3 ", x[3:])
print("even indeces ", x[::2] )
print("uneven indeces ", x[1::2]) ## starts at 1
print("reverse ", x[::-1] ) ## step value is negative starts at last element
```

Let us slice a Numpy array "x" with 15 values

[0 1 2 3 4 5 6 7 8 9 10 11 12 13 14]

The previous code listing gives us the following slicing results:

Listing 1.28: Examples of Slicing - output

```
first 4 elemets    [0 1 2 3]
all after 3        [ 3  4  5  6  7  8  9 10 11 12 13 14]
even indeces       [ 0  2  4  6  8 10 12 14]
uneven indeces     [ 1  3  5  7  9 11 13]
reverse            [14 13 12 11 10  9  8  7  6  5  4  3  2  1  0]
```

Slicing a sub matrix from a larger matrix can be done as follows:

Listing 1.29: Slicing a sub matrix from a larger matrix

```
## slicing
a = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9, 10, 11, 12]] )
print(a)
print(a[:2,:2])
```

Here, from a 3x4 matrix, we get a 2x2 matrix after slicing. This is shown in the next code listing.

Listing 1.30: Slicing a sub matrix from a larger matrix - output

```
## input

    [[ 1  2  3  4]
     [ 5  6  7  8]
     [ 9 10 11 12]]

## after slicing

     [[1 2]
      [5 6]]
```

That concludes are quick introduction to Numpy arrays. Now we are ready to move on to PyTorch operations on tensors.

1.4.2 Tensor Operations with PyTorch

In this section, we will now look at some special tensor operations and how they can be done with the PyTorch framework.

First we call the PyTorch library.

Listing 1.31: Libraries

```
import torch
import numpy as np
import matplotlib.pyplot as plt
```

Let us begin our discussion of tensor operations with some simple examples defining tensors in PyTorch.

Listing 1.32: Defining Torch Tensors

```
a_tr = torch.ones(5)

print( a_tr )

print(    float(    a_tr[1]    )       )

a_tr[2]  =  1456.0

print(  a_tr  )
```

Our previous code creates a torch tensor of size 5 and we then modify the third value of the tensor to 1456.0.

The function **torch.argmax()** is a PyTorch function that returns the index of the largest value across the axis of a tensor. For example, in the code below we select the index for the highest values for 2 tensors, respectively. As can be seen in the code listing below, answer1 is equal to 2, and answer2 is equal to [2, 0].

Listing 1.33: Torch.argmax

```
answer1 = torch.argmax( [35, 4, 72, 2] )

## answer1  =          2


answer2 = torch.argmax( [  [23, 32, 49],
                           [45,  1, 12]  )

## answer2  =    [2, 0]
```

Squeezing and reshaping are very important operations in PyTorch. Let us look at an example. First, we create a dummy tensor of size [64, 1, 28, 28]

Listing 1.34: Create a Torch tensor

```
batch_dummy_torch = torch.rand(64, 1, 28, 28)
batch_dummy_torch.shape
```

to convert it to a size [64, 28, 28] we can use the **torch.squeeze** function as follows:

Listing 1.35: Torch squeeze

```
batch_dummy_torch = torch.squeeze(batch_dummy_torch, dim=1)
batch_dummy_torch.shape
```

Now we will proceed to reshape the part of the tensor that is 28x28 into just one dimension equal to 784. We can do that with the following code which will return a tensor of size [64, 784].

Listing 1.36: Torch reshape

```
batch_dummy_torch = batch_dummy_torch.reshape((-1, 784))
batch_dummy_torch.shape
```

Here the -1 is used to infer the other dimensions.

Neural networks have the general form:

$$y = w * x + b$$

In the next code segment we will initialize the tensors **w** and **x** and perform a matrix multiplication.

Listing 1.37: matmul w * x

```
x = torch.randn(150, 4)
x.shape

## torch.Size([150, 4])

w = torch.randn( 4, 6 )
w.shape

## torch.Size([4, 6])

h1 = torch.matmul(  x, w  )
h1.shape

## torch.Size([150, 6])
```

Neural network connections from one layer such as **"x"** to the next layer such as **"h1"** can be thought of in terms of the previous matrix multiplication. This operation and concept can be seen in the following figure.

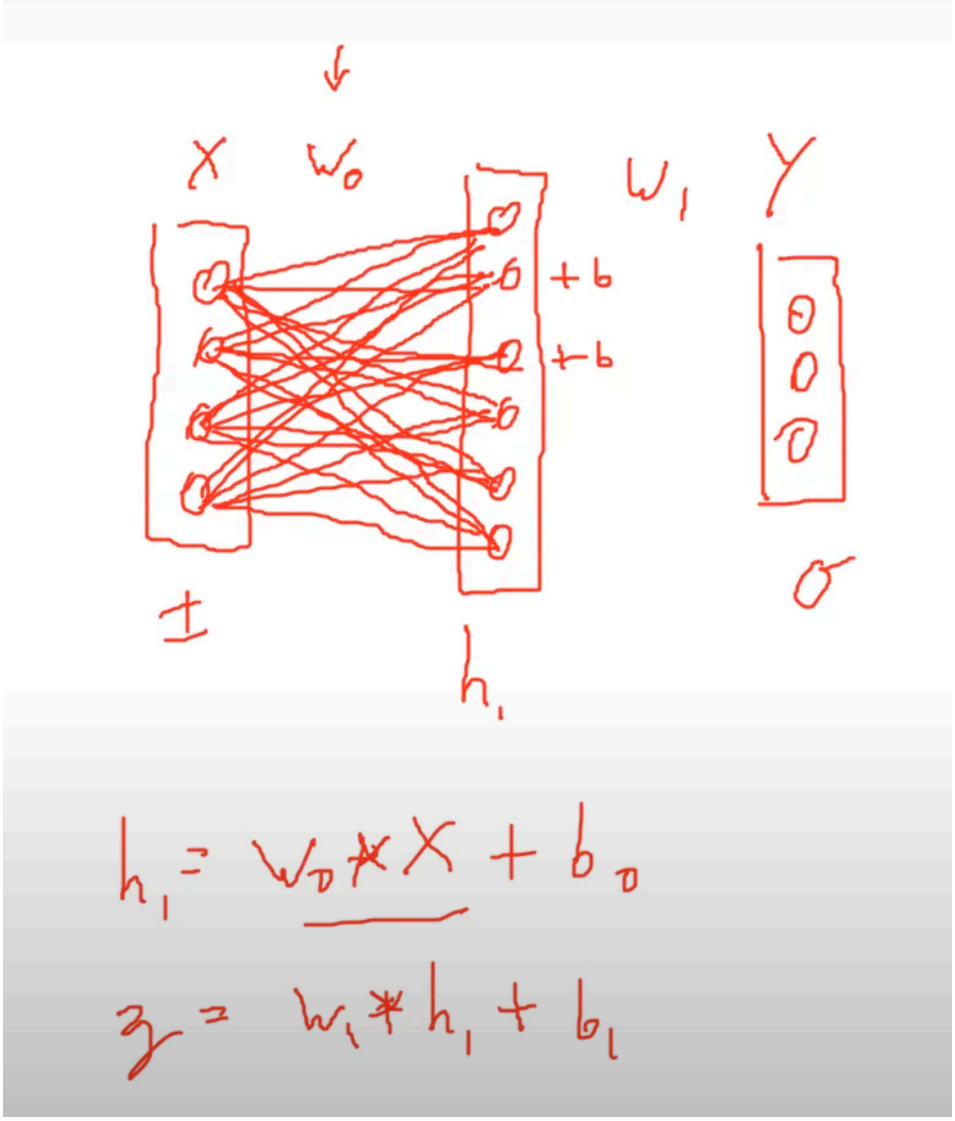


Figure 1.2: A matrix multiplication as a neural network

1.5 Conclusion

This concludes chapter 1. In it, I described some of the background on the deep learning tools and the development environment needed, as well as many math operations with tensors.

2 Traditional Machine Learning

In this chapter, I will address some of the traditional machine learning topics. I will use python, numpy, and the Sklearn library to provide example code on how to use the different models and other tools. I will quickly move through logistic regression to arrive at other algorithms. Much of what we learn from using the SKlearn tool kit can be integrated with PyTorch to make your deep learning code more powerful and modular. That way, you will be able to re-use your code for many different tasks. I will not focus on the theory of traditional machine learning algorithms here. There are many books on the theory of machine learning algorithms so instead I will concentrate on the practical aspects of using them in Python.

Objective: To understand Traditional Machine Learning techniques.

Machine Learning (ML) is essential to enable automated systems to make decisions and to infer new knowledge about the world. This section describes some of the most important traditional methodologies currently in use in the field of machine learning. Machine learning approaches can be divided into supervised learning (such as Support Vector Machines) and unsupervised learning (such as K-means clustering). Within supervised approaches, the learning methodologies can be divided, based on whether they predict a class or a magnitude, into classification and regression approaches, respectively. An additional categorization for ML methods depends on whether they use sequential or non-sequential data.

Classifiers are machine learning approaches that produce as an output a specific class given some input features. Important classifiers include Support Vector Machines (Burges 1998) commonly implemented using LibSVM (Chang and Lin, 2001), Naïve Bayes, artificial neural networks, deep learning based neural networks, decision trees, random forests, and the k-nearest neighbor classifier (Witten and Frank, 2005).

Regression models are those that produce a real valued number (magnitude) instead of a class such as the price or square footage of a house, or temperature, etc.

In their simplest form, deep learning based methods are simply neural networks with more layers between inputs and outputs. Deep learning methods have made a big impact in the field of machine learning in recent years. In particular, they are very important because, given enough computational power, they can automatically learn the optimal features to be used in a ML model from the raw data. In the past, learning what features to use required using

humans to engineer the features. This issue has now been alleviated considerably by deep learning. Additionally, artificial neural networks are models that can handle non-linearly separable data. In theory, this capability allows them to model data that may be more difficult to infer.

2.1 Code Issues

Before we begin, I want to address some issues about the code. First, I have tried to be consistent with other programmers of machine learning in Python. We will be using many libraries to implement the models. In particular, I selected to use the SKLearn library since it is very powerful and widely use. Especially for pre-processing steps.

In general, I only introduce enough SKlearn code for us to know how to integrate it with PyTorch in our deep learning endeavors. For a more in-depth discussion of SKlearn, I highly recommend the book "Python Machine Learning" by Sebastian Raschka. I have tried to be consistent with the conventions used in that book so that both books can be used together.

So, now, getting back to the code, here is an example of some of the most important libraries that we can use for our machine learning modelling.

Listing 2.1: Libraries

```
import numpy as np
from sklearn import datasets
from sklearn.cross_validation import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from matplotlib.colors import ListedColormap
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score, f1_score

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn import decomposition
```

As you may imagine the SKlearn library is the main library which contains most of the tradi-

tional machine learning tools we will discuss in this chapter. The numpy library is essential for efficient matrix and linear algebra operations. For those with experience with MatLab, I can say that numpy is a way of performing linear algebra operations in python similar to how they are done in MatLab. Using matrix and linear algebra operations makes the code more efficient in its implementation and faster as well. The datasets library seen above helps to obtain standard corpora (datasets). You can use it to obtain annotated data like Fisher's iris data set, for instance.

From **sklearn.cross_validation** we can import **train_test_split** which is used to create splits in a data matrix such as using 70 % for training purposes and 30 % for testing purposes. From **sklearn.preprocessing** we can import the **StandardScaler** module which helps to scale data. We can use functions such as these to scale our data for the PyTorch based models as well. Deep learning algorithms can improve significantly when data is properly scaled. So, it is recommended to do this. We will use the **sklearn.metrics** module for performance evaluation of the classifiers. I will show that this module can be used with SKlearn models and with PyTorch models. Again, this will help to more easily understand deep learning since we do not have to use the more complex and very verbose PyTorch functions.

The main metrics used to evaluate classification models are:

- accuracy_score
- recall_score
- f1_score
- precision_score
- confusion_matrix

The main metrics used to evaluate regression models are:

- coefficient of determination (R^2)
- RMSE

Two more very important libraries are **matplotlib.pyplot** and Pandas. The **matplotlib.pyplot** library is very useful for visualization of data and results, and the Pandas library is very useful for pre-processing. The Pandas library can be very useful to pre-process

large data sets in very fast and very efficient ways. There are some parameters that are sometimes useful to set in your code. The code sample below shows the use of **np.set_printoptions**, for instance. The function is used to print all values in a Numpy array. This can be useful when trying to visualize the contents of a large data set.

Listing 2.2: Print options

```
## set parameters

np.set_printoptions(threshold=np.inf)

## print all values in numpy array
```

Let us assume that our data is stored in the matrix **"X"**. The code segment below uses the function **train_test_split**. This function is used to split a data set (in this case X) into 4 sets which are **X_train**, **X_test**, **y_train**, **y_test**. These are the 4 sets that will be used by the traditional or the deep learning models. The sets that start with **"X"** hold the input data (feature vectors) and the sets that start with **y** hold the output labels per sample (e.g. y_1 for the first feature vector, y_2 for the second feature vector, and so on). The values test_size=0.01 and random_state=42 in the function are parameters that define the split. The value 0.01 makes a train set that has 99 percent of all samples while the test set has 1 percent of all samples. In contrast, test_size=0.20 would mean that there is an 80 % and 20 % split. The random_state=42 allows you to always get the same random data since the seed is defined as 42.

Listing 2.3: Split the data

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30,
    random_state=48)

## k-folds cross validation all goes in train sets (hence 0.01)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.01,
    random_state=42)
```

To call all the functions or models you can employ the following approach. Here we have defined 5 common models. Notice that each one gets the 4 data sets obtained from the percentage split operation. Notice also that the data files have a **_normalized** added to their

name.

This is a good standard approach used by programmers to indicate that this data has been scaled (the next chapter addresses scaling). Here you run **X_train** through a scaler function to obtain **X_train_normalized**. The labels **y** are not scaled, in this case. Although they can be if needed.

Listing 2.4: Sklearn ML models

```
logistic_regression_rc(X_train_normalized, y_train, X_test_normalized, y_test)

svm_rc(X_train_normalized, y_train, X_test_normalized, y_test)

random_forest_rc(X_train, y_train, X_test, y_test)

knn_rc(X_train_normalized, y_train, X_test_normalized, y_test)

multilayer_perceptron_rc(X_train_normalized, y_train, X_test_normalized, y_test)
```

Before we talk about the models, let us address object oriented programming and performance evaluation. Performance evaluation will help you to determine how good your classifier (model) is given an annotated test data set.

2.2 Object Oriented Programming

Object Oriented Programming (OOP) will be used extensively in this book to implement our ML and deep learning (DL) algorithms. As is necessary with progress, the DL algorithms are more complicated, with deeper and more resource intensive networks. This is best exemplified by one of the newest deep learning algorithms: The Transformer. The NN algorithms are just much more complicated. A little bit too much in fact and some proprietary programming libraries, as a result, are starting to abstract too much of the code. I have found that a simple solution to continue to write from scratch code is to write everything in an object oriented fashion. This way you are still defining the classes from scratch but you can also build far more complicated algorithms thanks to traditional object oriented programming techniques. As we will see later, a GPT can be nicely built with a reasonable amount of lines of code if leveraging object oriented programming techniques.

The following code segment includes a simple example of an OOP class format we will use extensively throughout this book for defining DL models.

Listing 2.5: Object Oriented Programming

```
class LinRegNet(nn.Module):

    ## init the class
    def __init__(self):

        super().__init__()

        self.linear1 = nn.Linear(11, 1)

    ## perform inference
    def forward(self, x):

        y_pred = self.linear1(x)

        return y_pred
```

In general, our DL model classes will include an **init** function and **forward** function.

2.3 Performance Evaluation

As previously stated, performance evaluation depends on whether we are doing classification or regression.

2.3.1 Regression Performance Evaluation

The main metrics used to evaluate regression models are:

- Coefficient of determination (R^2)
- RMSE

My favorite regression evaluation metric is R^2. The range for the metric goes from 0-1 where 1 means that you have a good model. Generally, I have used sklearn **metrics** to evaluate regression models. The code can be seen in the following code listing:

Listing 2.6: Print statistics - Regression

```
## coefficient of determination (R**2)
from sklearn.metrics import r2_score

print(
      r2_score( y_pred.detach().numpy(), y_test.numpy()      )
)
```

2.3.2 Classification Performance Evaluation

The main metrics to evaluate classification models are:

- accuracy_score
- recall_score
- f1_score
- precision_score
- confusion_matrix

Evaluation of the performance of your classifiers is extremely important. The SKlearn kit provides very good modules to address this issue. In particular, evaluation often involves measuring accuracy, precision, recall, and f-measure. The best way to understand these metrics is to think of a confusion matrix. Confusion matrices show how many elements from a class are correctly and incorrectly classified.

For example, take the following:

	Predicted 1	Predicted 0
True 1	a	b
True 0	c	d

Figure 2.1: Performance errors

Given this table, we can calculate accuracy as:

$$accuracy = (a + d)/(a + b + c + d)$$

Precision is defined as follows:

$$\text{precision} = a / (a + c)$$

The recall metric can be computed as follows:

$$\text{recall} = a / (a + b)$$

Finally, the ”f” measure, which is a harmonic average of recall and precision, can be computed as follows:

F = (2 * (precision*recall)) / (precision + recall)

The sample code to obtain these metrics using sklearn is provided below.

Listing 2.7: Print statistics - Classification

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score, recall_score, accuracy_score, f1_score

def print_metrics_function(y_test, y_pred):
    print('Accuracy: %.2f' % accuracy_score(y_test, y_pred))
    confmat = confusion_matrix(y_true=y_test, y_pred=y_pred)
    print("Confusion Matrix:")
    print(confmat)
    print('Precision: %.3f' % precision_score(y_true=y_test, y_pred=y_pred, average='
        weighted'))
    print('Recall: %.3f' % recall_score(y_true=y_test, y_pred=y_pred, average='
        weighted'))
    print('F1-measure: %.3f' % f1_score(y_true=y_test, y_pred=y_pred, average='
        weighted'))
```

The code above shows a function to print the performance metric statistics for a classifier. Notice that two sets are provided which are **y_pred** and **y_test**. The **y_test** data set contains the original annotated labels. The **y_pred** data set contains the labels predicted by your classifier. Each of the metric functions such as **f1_score** and **recall_score** uses these 2 data sets to calculate the respective metric. Generally speaking, the higher the better.

2.3.3 Plotting Performance

In the following code segment we can see the function **plot_metric_per_epoch()** which can be used to plot a metric value every N epochs. For this to work, you store the metric values in a list such as **precision_scores_list** and then proceed to plot these values using the **matplotlib** library.

Listing 2.8: Plotting Performance

```
def plot_metric_per_epoch( precision_scores_list ):
    x_epochs = []
    y_performance= []
    for i, val in enumerate(precision_scores_list):
        x_epochs.append(i)
        y_performance.append(val)
    plt.scatter(x_epochs, y_performance,s=50,c='lightgreen', marker='s', label='score
        ')
    plt.xlabel('epoch')
    plt.ylabel('score')
    plt.title('Score per epoch')
    plt.legend()
    plt.grid()
    plt.show()
```

The output should look like the following figure.

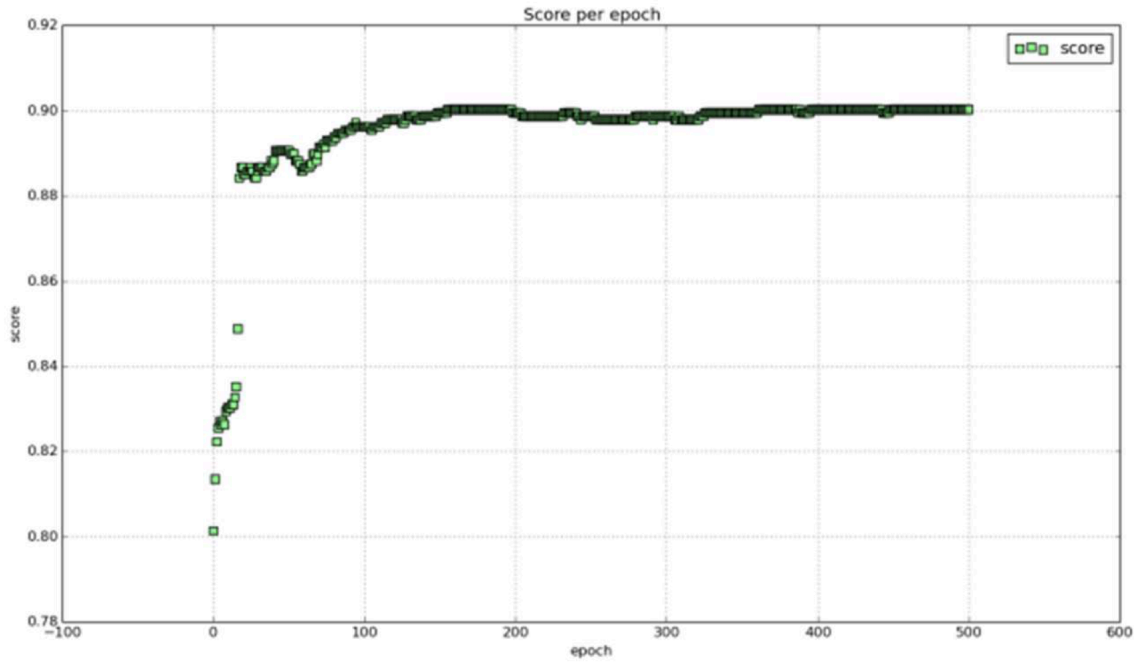


Figure 2.2: Performance per epoch example

2.4 Optimization

Before we begin to discuss some of the machine learning algorithms, I should also say something about optimization. Optimization is a key process in machine learning. Basically, any supervised learning algorithm needs to learn a prediction equation given a set of annotated data. This prediction function usually has a set of parameters that must be learned. However, the question is **"how do you learn these parameters?"**

The answer is that you do so through:

Optimization

In its simplest form, optimization consists of trying many sets of parameters with your model and seeing what result they give you. If the result is not good, the optimization algorithm needs to decide if you should decrease the values of the parameters or increase the values of the parameters.

In general, you do this in a loop (increasing and decreasing the parameter values) until you find an optimal set of parameters. But one of the questions to answer here as you are looping is: **do the values go up or down in this iteration?** Well, as it turns out, there are methodologies based on **calculus** that help you to make this decision.

Let us try to picture this with a graph (below).

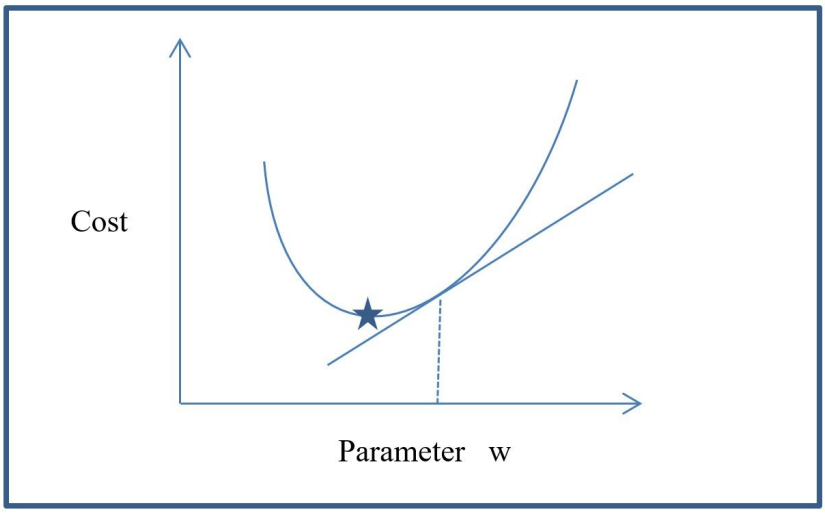


Figure 2.3: Optimization graph

The above graph represents an optimization problem. The **y** axis represents the cost (or penalty) of using a given parameter. The **x** axis represents the value of the parameter (**w**) being used at the given iteration. The curve represents the behavior that the function (loss) being used to minimize the cost will follow for every value of parameter **w**.

$$loss = (\hat{y} - y)^2$$

As shown in the graph, the optimal value for the curve is found where the star is located (i.e where the value of cost is at a minimum). So, somehow the optimization algorithm needs to travel through the function and arrive at the position indicated by the star.

At that point, the value of **"w"** reduces the cost and finds the best solution. Instead of trying all values of **"w"** at random, the algorithm can make educated guesses about which direction to follow (up or down). To do this, we can use **calculus** to calculate the derivative of the function at a given point. This will allow us to determine the slope at that point. In the case of the graph, this represents the tangent line to the curve if we calculate the derivative at point **w**. If we calculate the slope at the position of the star symbol, then the slope is zero because the tangent at that point is parallel to the **x** axis. The slope at the point **"w"** will be positive.

Based on this result, we can tell the direction we want to take for parameter **w** (decrease or increase). This type of optimization is called gradient descent and is very important in machine learning and deep learning. There are several approaches to implement gradient descent and this is just the simplest explanation for conceptual purposes. We can write the algorithm for this technique as follows:

Listing 2.9: Optimization Algorithm

```
old_x = 0
new_x = 4
step_size = 0.01
precision = 0.00001

def function_derivative(x):
    return 3*x ** 2 - 6*x

while absolute_value(new_x - old_x) > precision:
    old_x= new_x
    new_x = old_x - step_size * function_derivative(old_x)

print"(result is: ", new_x)
```

In the previous code example we assume a loss function of:

$$f() = x^3 - 3x^2 + 7$$

that needs to be optimized for parameter **x**. We will need the value of the derivative for each point x. The derivative for f() is:

$$f'() = 3x^2 - 6x$$

So, the parameter **x** can be calculated in a loop using the derivative function which will determine the direction to follow when increasing or decreasing the parameter **x**.

2.5 KNN

The k-nearest neighbor (KNN) classifier is a popular algorithm that I always like to use. It requires very little parameter tuning and can be easily implemented. Here, the KNN code is implemented in pure Numpy because of its simplicity. The **n_neighbors** parameter is the **k**. In this case, the **k** closest samples are selected.

In the next code listing, you can see all the code needed to run KNN. Wow! Pretty short right? This code splits our data into **train** and **test** sets. Then grabs every sample in the test set and compares it to every sample in the train set. For each test sample, we get all distances between the test sample and all train samples. We then rank the distances and select the closest **K** distances. Finally, we assign the majority class associated with the closest 5 distances. That is it!

First the libraries.

Listing 2.10: The libraries for the KNN algorithm code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.tri as tri
from pyodide.http import open_url
```

Now the KNN algorithm.

Listing 2.11: The whole KNN algorithm code

```
def euclidean_distance(v1, v2):
    return np.sqrt( np.sum(    (v1 - v2)**2    )    )

def predict(test_x):
    k = 5
    distances = [ euclidean_distance(test_x , x)  for x in X_train     ]
    k_neighbor_indices = np.argsort(distances)[:k]
    labels = [ y_train[i]  for i in k_neighbor_indices  ]
    labels_np = np.array( labels  )
    pred = int( np.mean(labels_np)  )
    return pred

url1 = ("https://rcalix1.github.io/Build-Fun-AI-Projects-that-Run-on-the-Web/volume
    -1-pyscript-and-knn/chapter3/knn/iris.csv")

iris_pd = pd.read_csv(open_url(url1))
iris_pd['species'] = iris_pd['species'].replace({'setosa':0, 'versicolor':1 , '
    virginica':2})
iris_np = iris_pd.to_numpy()

np.random.shuffle(iris_np)

X_train    = iris_np[1:130, :4]
y_train    = iris_np[1:130,  4]
X_test     = iris_np[130:150, :4]
y_test     = iris_np[130:150, 4]

def my_gen_function():
    accum_res = ""
    for i, test_x in enumerate( X_test ):
        the_pred = predict(test_x)


my_gen_function()
```

The KNN code includes a function called the Euclidean distance as can be seen in the next code listing.

Listing 2.12: Euclidean Distance

```
def euclidean\_distance(v1, v2):
    return np.sqrt( np.sum(    (v1 - v2)**2    )    )
```

This is the function that measures the distance between 2 points in a vector space. These 2 points need to have the same size but the size can be of any dimension. For instance, for our Iris data, every point has 4 features. So in the code the 2 points v1 and v2 would be of size 4 each. However, we could also have points (samples) of many more dimensions. For instance, points with 100 features. So in this case v1 and v2 would both need to have size 100. But the great thing is that the function for distance calculation would still work. That is the power of Numpy.

The name Euclidean distance comes from a Greek philosopher named Euclid. He is best known for putting together one of the earliest books on geometry. The book was so good for its time that the type of mathematics it discussed became known as Euclidean geometry.

So, where does this magical Euclidean distance function come from? Would you believe that it is related to an idea one of the great Greek philosophers (Pytagoras) is credited with? Pytagoras was before Euclid and is credited with coming up with the Pythagorean Theorem (bubble in the figure).



Figure 2.4: Pythagoras

The theorem states that given a triangle (see figures), the sum of the areas of the two squares on the legs (a and b in green) equals the area of the square on the hypotenuse (c in green).

If you look closely at the figure below, you can see that I have written down the connection between Pythagoras' theorem, and the Euclidean function we used in our code.

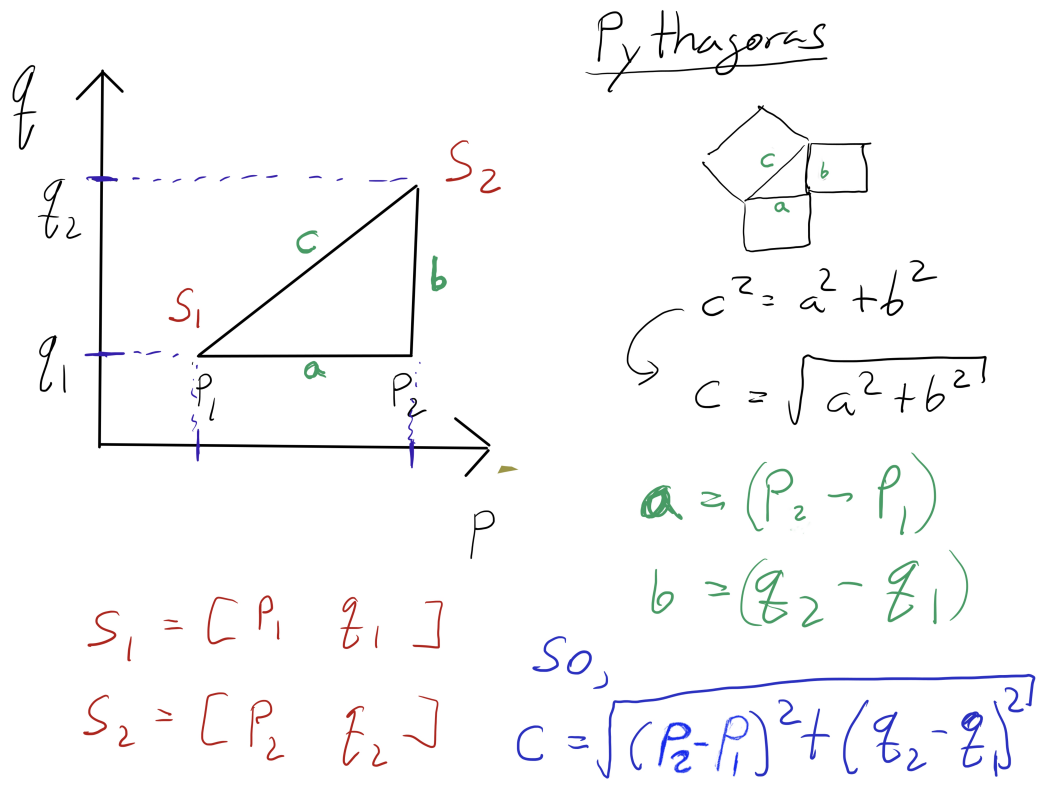


Figure 2.5: Pythagorean Theorem and the distance function

Now, with some of the math history out of the way, let us continue with our code description. In the next section we can see some of the details of the KNN **predict** function. Actually, the **predict** function is the actual KNN algorithm. I will describe it in detail next. The variable **test_x** is the sample in question. Say, one Iris test sample with 4 features (**[x1, x2, x3, x4]**). The next line of code:

```
distances = [ euclidean_distance(test_x , x) for x in X_train ]
```

calculates all the distances between **test_x** and all the training samples in **X_train**. The way this statement is written is called a list comprehension in Python. The **"for x"** part of the statement means that each sample in the train set is grabbed and passed to the **euclidean_dis-**

tance function along with the test sample. Both points are passed to the Euclidean equation and the distance between them is returned. This is done for every training sample and in the end, the list **"distances"** contains all the measured distances.

The next statement:

k_neighbor_indices = np.argsort(distances)[:k]

takes all the distances and, using **np.argsort()**, sorts them. Then we slice the sorted vector with **[:k]**. This slicing only returns the indeces for the k smallest distances. The indeces are assigned to the array named **k_neighbor_indices**. One important note is that **"argsort"** returns only the indeces in the vector and not the values themselves. Since the indeces in the **"X"** data and **"y"** labels are aligned, then we can use these same indeces to extract the corresponding labels from **"y"**. And that is exactly what we do with the following statement:

labels = [y_train[i] for i in k_neighbor_indices]

Finally, the list **labels** is converted into a Numpy array of the labels which are numbers, and the mean of them is calculated. That is it. You have found the label.

Wasn't that easy. We have described the whole KNN algorithm from scratch. Wow!

2.6 Logistic Regression

Logistic regression is a simple algorithm that is often used by practitioners of machine learning because it can obtain good results. Logistic regression is a linear function much like linear regression which predicts the probability of a sample belonging to a given class. Logistic regression uses another optimization function instead of the standard least squares cost function used in linear regression.

Listing 2.13: Log Reg

```
def logistic_regression_rc(X_train_normalized, y_train, X_test_normalized, y_test):
    from sklearn.linear_model import LogisticRegression
    lr = LogisticRegression(C=1000.0, random_state=0)

    lr.fit(X_train_normalized, y_train)
    y_pred = lr.predict(X_test_normalized)

    print_stats_percentage_train_test("log re", y_test, y_pred)
```

The predicted values from a standard regression approach are now passed through a sigmoid function that basically maps the output to a probability range scale between 0 and 1. The code above provides an example of how to use the logistic regression function with SKlearn. Later, we will implement this logistic regression function again with PyTorch from scratch.

In the previous function, the train and test sets are provided for the model to be trained and tested. In SKlearn most steps are abstracted. In contrast, PyTorch will allow us to define more steps such as the cost function, optimization, and inference equation, and other aspects. In the function **logistic_regression_rc**, first you initialized a logistic regression object (lr) and then you train and test it with the functions **lr.fit** and **lr.predict**. The final step is to measure performance using the previously described function **print_stats_percentage_train_test**. Most classifiers are implemented in the same way with SKlearn. In the next section, I will demonstrate how this is done for a neural network in Sklearn.

2.7 Neural Networks

Neural networks can be very complex systems that take a long time to train. Therefore, the use of them in SKlearn may not be recommended except for the smallest of data sets. The code is shown here for contrast purposes with later implementations of neural networks in PyTorch. In the next chapters, we will focus on how to do this in PyTorch and how to create networks of multiple layers from scratch.

In the code below we can see that everything is very similar to the previous logistic regression implementation. The new changes appear in the definition of the **clf** multilayer object. Here, the parameter **hidden_layer_sizes**=(100,100) means that the architecture of the network

consists of 2 hidden layers with 100 neurons each. A parameter such as (200,) would mean that the network has 1 hidden layer with 200 neurons.

Listing 2.14: NN in sklear

```
def multilayer_perceptron_rc(X_train_normalized, y_train, X_test_normalized, y_test):

    from sklearn.neural_network import MLPClassifier

    clf = MLPClassifier(solver='lbfgs', alpha=1e-5, hidden_layer_sizes=(100,100),
        random_state=1)

    clf.fit(X_train_normalized, y_train)
    y_pred = clf.predict(X_test_normalized)

    print_stats_percentage_train_test("multilayer perceptron", y_test, y_pred)
```

2.8 Support Vector Machines (SVM)

Before Deep Neural Networks (DNNs), SVM was one of the most important ML algorithms. SVM is an example of a theoretically well founded machine learning algorithm. And for this reason it has always been very well respected by machine learning practitioners. This is in contrast to neural networks which haven't always been considered as being very strong in regards to their theoretical framework. As an example, in the rest of this section I will discuss SVM's framework and then provide example code to implement an SVM classifier with SKlearn. Support Vector Machines is a binary classification method based on statistical learning theory which maximizes the margin that separates samples from two classes (Burges 1998; Cortes 1995). This supervised learning machine provides the option of evaluating the data under different spaces through Kernels that range from simple linear to Radial Basis Functions [RBF] (Chang and Lin, 2001; Burges 1998; Cortes 1995). Additionally, its strong theoretical framework and ability to handle large feature spaces use to make it an attractive tool for many applications in ML. Statistical Learning Theory (SLT) methods assume prediction models that can be ascribed a confidence characteristic. They are based on the fact that both structural and empirical risks are minimized. The expected risk can be calculated based on the empirical risk that is present in the data with the associated upper and lower bounds.

The code for SVM using sklearn can be seen in the next code listing.

Listing 2.15: SVM

```
def svm_rc(X_train_normalized, y_train, X_test_normalized, y_test):
    from sklearn.svm import SVC
    #svm = SVC(kernel='linear', C=1.0, random_state=0)
    svm = SVC(kernel='rbf', random_state=0, gamma=0.0010, C=32)

    svm.fit(X_train_normalized, y_train)
    y_pred = svm.predict(X_test_normalized)

    print_stats_percentage_train_test("svm (rbf)", y_test, y_pred)
```

In SVM, the maximization of the margin is based on the training samples that are closest to the optimal line (also known as support vectors). Because the method tries to maximize the margin between the samples of two classes under a set of constraints, it ultimately becomes an optimization problem to find the maximum separation band. The function that represents the margin is quadratic and can be solved using quadratic programming techniques with Lagrange operators.

Non-linearly separable cases can be solved by mapping the initial set of features to a higher feature space by way of a Kernel Trick. This will provide higher freedom in separating the data in higher dimensional space. The Kernel trick takes advantage of the fact that SVMs do not need to know the mapping function because this is expressed as the dot product of the input data.

Since most real world data includes outliers and noise, a soft margin approach can be introduced in the model to allow for some errors to occur. This softening of the margin is achieved by introducing an error term where the cost represents the penalty for each error

Unlike this beautifully well-defined algorithm, neural networks and deep neural networks are not always considered to be so well founded in theory. Instead, over the years they have been considered as a type of technique with a black box framework. Because of this, many ML practitioners have at times disregarded neural networks and deep neural networks. However, since the early 2000s, deep neural networks have started to defeat all other techniques on challenge after challenge across the machine learning landscape. As such, both academia and industry have noticed and great interest has developed for these techniques.

Similarly to the previous algorithms, SVM has the same structure as others in SKlearn. It

includes a parameter kernel which can be used to set kernels such as **linear** or **rbf**. The **rbf** kernel requires the parameters of cost and gamma. These values are usually data dependent and require parameter tuning.

2.9 Regression, Trees, and XGBoost

There are several ML approaches to fit models to regression data. Three important ones are linear regression, regression trees, and neural networks. The next section discusses regression and regression trees leading up to XGBoost. A more detailed discussion of regression will be covered later in the book.

2.9.1 Regression Trees

Regression trees (Torgo 2017) can fit nonlinear regression data. Regression trees are a type of decision tree in which the predicted values are not discrete classes but instead are real valued numbers. The key concept here is to use the features from the samples to build a tree. The nodes in the tree are decision points leading to leaves. The leaves contain values that are used to determine the final predicted regression value.

2.9.2 XGBoost

The XGBoost (Chen et al.) techniques represent an improvement over other regression techniques like gradient boosting. XGBoost has new algorithms and approaches for generating the regression tree. It also has several optimizations that improve the speed and performance by which it can learn and process data. As of 2023, XGBoost can outperform many ML techniques including neural networks for small tabular data.

2.10 The t-test for Machine Learning

Sometimes we want to compare between several machine learning algorithms using the previously discussed metrics (precision, recall, f-measure). The standard approach is to run 10-fold cross validation and look at the results. However, a more robust approach is to use a t-test. By definition, the t-test compares the mean or averages of 2 populations to determine how different the populations are from each other.

2.11 Summary

This chapter provided an overview of some of the main traditional topics in machine learning. In particular, the following machine learning algorithms were presented: logistic regression, KNN, Support Vector Machines, and neural networks. Code examples of their implementation using the sklearn toolkit or Numpy were presented and discussed. Additionally, issues related to classifier performance were also addressed. The next chapter will focus on issues related to data and data pre-processing that apply to both traditional machine learning as well as deep learning.

3 Data Loading and Pre-processing

In this chapter, I will address the very important issue of dealing with the data. To me and most practitioners, data collection is the most important issue in machine learning.

Objective: To address Data Loading and Pre-processing.

There are many aspects that must be addressed when dealing with data. These include:

- getting the data
- cleaning the data
- pre-processing the data
- building a corpus and annotating it
- performing inter-annotator agreement
- etc.

Here in this chapter I will address general issues about data as well as data issues related to deep learning such as one-hot encoding.

3.1 Loading the Data

Data can be obtained from the web such as text from twitter or web pages. Specific data sets can also be obtained from the machine learning libraries. Sklearn has a "dataset" module. This dataset module can be used to obtain certain data sets such as the Iris dataset. An example of this code can be seen below.

Listing 3.1: Iris data

```
iris = datasets.load_iris()

X = iris.data[:, [1,2,3]]

y = iris.target
```

Here, the first index in the data matrix represents the rows and the second index represents the columns. Data can also be obtained from text files. Many practitioners and academics will have their own data in text files. This data can be formatted in many different ways and loaded into the code. In most of the examples in this book, the data is assumed to be formatted in csv (comma separated) format. The code to load the data to both SKlearn based traditional machine learning algorithms and to PyTorch code files is shown below. In the code we can see that we can use the Numpy library (or namespace) to obtain the data. For the example below we assume that the data is stored in the file **data/12559_Training_Dataset.csv** and is read by loadtxt() into the python variable **Matrix_data**. Since we are using csv format, the Iris data file can look like the following.

Listing 3.2: Iris data

```
1.0,6.1,2.8,4.7,1.2
0.0,5.7,3.8,1.7,0.3
2.0,7.7,2.6,6.9,2.3
1.0,6.0,2.9,4.5,1.5
1.0,6.8,2.8,4.8,1.4
0.0,5.4,3.4,1.5,0.4
...
```

Once the data is in **Matrix_data**, it can be processed as a Numpy array matrix. This means that it is no longer just an array but instead it is more a vector or matrix as in linear algebra. Many operations are now simplified like extracting certain columns or rows. This is usually referred to as slicing.



Figure 3.1: Like slicing a cake

Listing 3.3: Iris data slicing

```
f_numpy = open("data/12559_Training_Dataset.csv",'r')

Matrix_data = numpy.loadtxt(f_numpy, delimiter=",", skiprows=1)
A = len(Matrix_data[0,:])
print("num features,", A )

#X=Matrix_data[:, [1,2,3,4,5,6]]

X = Matrix_data[:,:18]                #[:,:149]
y = Matrix_data[:, 19]
```

In the previous code we can see that we can calculate the dimensions of the matrix as follows.

A = len(Matrix_data[0,:])

And this gives you the number of columns or the number of features plus the class.

3.2 Data and Feature Pre-Processing

Feature scaling is very important to achieving good results in ML modeling tasks. For example, in Principal Component Analysis (PCA) which is a feature reduction technique, feature scaling is very important. The purpose of PCA is to project data to a space that captures the most variability in the data. If the features are not scaled properly, one feature could dominate over the others and therefore be considered as the most variable feature. With feature scaling, features with real valued numbers from any range can be mapped to other ranges such as from -1.0 to 1.0. This is performed for all features so that no one feature will dominate in the model. Other classifiers and Deep learning models are also susceptible to feature scaling. The code below shows how the data can be scaled from **X_train** to **X_train_normalized**.

Listing 3.4: Feature Scaling

```
## feature scaling
sc = StandardScaler()

sc.fit(X_train)

X_train_normalized = sc.transform(X_train)
X_test_normalized  = sc.transform(X_test)
```

In the following code listing I am showing you a general way in which you can take a standard dataset such as Iris or mnist and save it to a csv file. This approach allows you to do the deep learning modeling by reading data from text files where the data is formatted in a very standard and well known format such as csv (comma separated format).

Listing 3.5: create csv files

```
## create csv files from mnist or iris

def buildDataFromMnist(data_set):
    #iris = datasets.load_iris()
    X_train, X_test, y_train, y_test =
                train_test_split(data_set.data, data_set.target, test_size=0.30,
                    random_state=42)

    f=open('2.0_training_mnist.csv','w')
    for i,j in enumerate(X_train):
        k=np.append(np.array(  y_train[i]), j    )
        f.write(",".join([str(s) for s in k]) + '\n')
    f.close()

    f=open('2.0_testing_mnist.csv','w')
    for i,j in enumerate(X_test):
        k=np.append(np.array( y_test[i]), j    )
        f.write(",".join([str(s) for s in k]) + '\n')
    f.close()
```

3.3 One Hot Encoding

DL algorithm implementations in PyTorch use one-hot encoding. Quite simply, one hot encoding means that you take labels in the following format.

Listing 3.6: one-hot encoding (before)

```
0
1
1
2
0
1
1
2
1
...
```

and convert them to labels in the equivalent one-hot encoded format as shown below. So, we create new vectors where all values are zero except for the value of the position of the

correct class in the vector.

Listing 3.7: one-hot encoding (after)

```
1,0,0
0,1,0
0,1,0
0,0,1
1,0,0
0,1,0
0,1,0
0,0,1
0,1,0
...
```

One-hot encoding transforms the labels vector (**y**) of size **n** into a matrix of size **n** by **B** where **B** represents the number of classes in the data set. For the case of the Iris dataset, we have 3 classes and, therefore, for a sample with label 2 we would get an equivalent one-hot encoded vector equal to [0,0,1]. The code to convert the data to one-hot encoded format is provided below. There are several ways of implementing one-hot encoding. I have used the approach below because I think it is the easiest to understand (while possibly not the most efficient or pythony). Notice how **"a"** is a 2-dimensional array initialized with all zeros. You use the value in the input vector data to determine the index **i** and **j** that are used to assign a 1 to the correct position in the matrix.

Listing 3.8: one-hot encoding code

```
# Convert to one hot data
def convertOneHot_data2(data):
    y=np.array([int(i) for i in data])
    #print y[:20]
    rows = len(y)
    columns = y.max() + 1
    a = np.zeros(shape=(rows,columns))
    #print a[:20,:]
    print rows
    print columns
    #rr = raw_input()
    #y_onehot=[0]*len(y)

    for i,j in enumerate(y):
        #y_onehot[i]=np.array([0]*(y.max() + 1) )
        #y_onehot[i][j]=1
        a[i][j]=1

    return (a)
```

3.4 Features

A supervised machine learning algorithm is only as good as the features that are provided to it. This statement used to be very true and many people made careers of just developing features for problems in different domains. For instance, in NLP, many people would spend a lot of time developing parsers and other techniques to find and create features from a text based problem. Similarly, in image processing, researchers developed many techniques to filter data out of images to perform efficient image classification. Today, deep learning has somewhat changed this. DL has managed to introduce approaches to decrease the amount of human involvement in the feature extraction process. Basically, deep learning methods, in some cases, have the ability to extract features from raw data using only un-supervised or semi-supervised techniques, transfer learning, etc. In some way, you can say that deep learning algorithms can extract the features themselves without human involvement. This ability has had a very strong impact in the performance of the algorithms implemented in industry and in the work performed by machine learning specialists. These abilities for enhanced feature extraction are available in the main mediums of text processing and image processing.

3.4.1 Features from Text

Feature extraction from text usually involves the processing of text documents to extract tokens, words, chunks, or other phrases to be used to create the features. There are many, many types of features that can be extracted from text. Some of the methods (Jurafsky and Martin, 2008) that can be used are:

- the bag of words approach
- frequency histograms of the words
- Part of speech tagging of the words
- Syntactic parsing of sentences
- Anaphora resolution
- word embeddings (the main method used today)

With the information provided by the previous methods, many features can be extracted. In general, text features can be binary or numeric. Binary features include the presence or absence of words, part of speech tags, chunks, syntactic parses, semantic parses, etc. Numeric based text features can be derived from performing counts or calculating distance metrics between words or higher level semantic concepts.

One simple technique, for example, that has had success in extracting features for supervised machine learning is called the gramulator (McCarthy et al. 2012). The gramulator is a feature extraction technique used particularly in natural language processing. The main idea is that, for a 2 class problem, you want to extract features (e.g. words or tokens) that are very frequent in one class but infrequent in the other. This helps to better discriminate between the classes. The downside of this approach, however, is that it needs a lot of annotated or labeled data to extract the grams or words from each class that are infrequent in the opposite class. If the grams are representative of the entire population; then, it can be expected that a classifier will have good performance in the classification task.

The downside of most of these techniques is that in the past they have required annotated data. Deep learning has several new techniques that address this issue and obtain good performance without needing large amounts of annotated data. These mainly involve some type of word embedding. Word embeddings are vector representations of words or syllables or subwords. The vectors are dense and usually of fixed size such as 256. The values of each vector representing the token in question are learned through some algorithmic scheme such

as Word2Vec (Mikolov et al. 2014), or Transformers (covered in later chapters).

3.4.2 Features from Images

Feature extraction from images is another area where deep learning is revolutionizing the way features are engineered and obtained. The input to a classifier when performing image processing and classification is an image. In general images are 2 dimensional arrays that contain the pixel intensities that define the image. For color images, you have 3 two-dimensional matrices where each stores the intensity for the R, G, and B color channels of the image.

Traditional Image Feature Extraction Techniques

In the past, these images where converted into feature vectors for use in machine learning using image processing based feature extraction techniques. Again, here, researchers spent considerable amounts of time performing feature engineering. Some of the methods widely used in image processing (Gonzales and Woods, 2007) include:

- Image segmentation
- Edge detection
- Region growing
- Image morphing
- Fourier transformations
- Etc.

Many features could be extracted using these methods. Generally speaking, many of these techniques are implemented as a filter that is applied to the images to convert them into another matrix of the same size or of another size. This new image will store new information about the image such as the pixel positions where an edge was detected. Here, once again, deep learning changed how this process is done.

Deep neural networks can be used to discover the features simply by providing the raw inputs and some annotation (e.g. labels). This information alone allows the neural network architecture to connect the neurons in ways where it can discover features by itself. In fact, deep neural networks are not just used for classification anymore but can also be used just for feature extraction. ResNet is an example of this. The extracted features can then be used

with other classifiers such as SVM, logistic regression, etc. In these cases, the input layer and hidden layers are used for feature extraction without having to use the output layer for the original classification task. Instead, the classification task can be changed to another task. Convolutional Neural Networks (CNNs) can provide this functionality for image processing and classification.

A Class to Load Images into Tensors

Loading images into tensors can be challenging. The following code listing presents an object oriented approach to load images from files into PyTorch tensors. This OOP class was written by my students Danielle Turner, David Higley, and Joseph Shapiro and I thank them for it.

First we import the libraries.

Listing 3.9: Load Images Class - libraries

```
import os
import torch
import imageio
import torchvision

from torch.utils.data import Dataset

from torchvision import transforms

from sklearn.model_selection import train_test_split
```

Next we define the class to load the images from folders into tensors.

Listing 3.10: Load Images Class - part 1

```
class ImageDataset(Dataset):

    def __init__(self, dataset_save="data.pt", raw_data=None, train=True, shuffle=
        False, transform=None, target_transform=None, convert=False, size=32):

        self.targets = []
        self.labels = []
        self.data = []

        self.X_train = []
        self.X_test = []
        self.y_train = []
        self.y_test = []

        self.transform = transform
        self.target_transform = target_transform

        if convert:
            self.convert(dataset_save, raw_data, size)
        else:
            self.load(dataset_save)

        seed = int(random.random() * 100) if shuffle else 42

        self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(self.
            data, self.targets, test_size=0.2, random_state=seed)

        if train:
            self.data = self.X_train
            self.targets = self.y_train
        else:
            self.data = self.X_test
            self.targets = self.y_test

    def __len__(self):
        return len(self.data)

    ...
```

The Dataset of images is in **data_path** which is the path to the images folder. This should contain multiple folders of each class.

Listing 3.11: folder of images structure

```
- images
    - cats
        - cat1.jpg
        - cat2.jpg
    - dogs
        - some-dog.jpg
    - ...
```

The current working directory is where the **pt** file will be stored. The **train** parameter is for whether to load train or test data. The **transform** parameter indicates the Torch transform to apply to the image data. The **target_transform** is the transform to apply to the targets if needed. The **convert** parameter is used to indicate if the data should be converted from images, or loaded from a **pt** file. The **size** parameter is to set the size of the images to convert to. This should be the same as the size of the images in the **pt** file (e.g. 226).

The next code segment includes the definitions of the **convert** and **load_functions** for the images.

The **convert** function in this OOP class is what convert the images to a defined size and then **center** crops.

Listing 3.12: Load images class - part 2

```
class ImageDataset(Dataset):

    ...

    def __getitem__(self, index):
        image, target = self.data[index], self.targets[index]
        if self.transform is not None:
            image = self.transform(image)
        if self.target_transform is not None:
            target = self.target_transform(target)
        return image, target

    def convert(self, dataset_save, raw_data, size):
        dataset = []
        self.labels = []
        targets = []
        if not os.path.exists(raw_data):
            raise ValueError('Raw image directory does not exist.')
        for folder in os.listdir(raw_data):
            if folder == ".DS_Store":
                continue

            for image in os.listdir(os.path.join(raw_data, folder)):
                if folder not in self.labels:
                    self.labels.append(folder)
                targets.append(self.labels.index(folder))

                img_arr = imageio.imread(os.path.join(raw_data, folder, image),
                    pilmode="RGB")
                resize = torchvision.transforms.Resize(size)
                crop_center = torchvision.transforms.CenterCrop(size)

                img = torch.from_numpy(img_arr).permute(2, 0, 1).float()
                img = resize(img)
                img = crop_center(img)
                img /= 255

                dataset.append(img)

        self.data = torch.stack(dataset)
        self.targets = torch.Tensor(targets).type(torch.LongTensor)

        torch.save((self.data, self.targets, self.labels), dataset_save)

    def load(self, dataset_save):
        if not os.path.exists(dataset_save):
            raise ValueError('Dataset file does not exist. Try creating the dataset
                by running with convert=True first.')
        self.data, self.targets, self.labels = torch.load(dataset_save)
```

3.5 Corpora (i.e. Annotated Datasets)

A supervised machine learning algorithm is only as good as the data that is provided to it. ML models learn how human annotators assign the labels to a sample and a system can only be expected to be as good as the human annotator. Some tasks are more subjective than others for human annotators and this is usually reflected in the classifier performance. Therefore, many practitioners recommend, before measuring classifier performance, that an analysis of the subjectivity of the human annotation should be performed. This is usually referred to as Inter Annotator Agreement. When annotating (labeling) a new resource, the quality of the annotation process must be measured in some way. Inter-annotator metrics refer to techniques used to measure the overall agreement between the annotations of two or more individuals (humans). Important metrics used to evaluate inter-annotator agreement (Artstein and Poesio, 2008) include:

- Average observed agreement
- Pi
- Alpha
- S
- Kappa

In general, these metrics differ in how they correct for expected chance agreement.

Expected chance agreement

Expected chance agreement is a probability that 2 annotators will agree on their annotation for an item by chance. This probability depends on the number of classes. Formally, this probability is calculated as follows:

$$A_e = \sum_{k \in K} P(k|c_1) \cdot P(k|c_2)$$

where c_i is the annotator i , and k is the assigned category. Inter-annotator agreement metrics are important because they help to set theoretical boundaries on the accuracy that a given machine learning methodology can achieve using the annotated corpora (Bird, Klein, and

Loper, 2009). A brief description of some of the techniques is provided in the following discussion.

Average Observed Agreement (Ao)

Averaged observed agreement is the easiest metric to compute. It is the percentage of annotations that two annotators agreed upon. The metric is formulated as follows where the variable **"samples"** represents the total number of annotation samples, and **"agreed"** is the amount of samples for which both annotators agreed.

$$A_o = \frac{1}{samples} \sum Agreed$$

Chance-corrected Metrics (Acorr)

Chance corrected metrics are those that take into account the **expected** chance agreement **Ae**. Once chance agreement is defined, the metric can be corrected. These types of metrics include: **s**, **alpha**, and **kappa**. Formally, the main concept in these metrics is defined as follows:

$$A_{corr} = \frac{A_o - A_e}{1 - A_e}$$

The following code segment shows how to perform this analysis using the well known **nltk** framework (www.nltk.org).

Listing 3.13: Inter-Annotator Agreement code

```
#interannotator agreement
import nltk

toy_data = [
    ['1', 5723,    'ORG'],
    ['2', 5723,    'ORG'],
    ['1', 55829,   'LOC'],
    ['2', 55829,   'LOC'],
    ['1', 259742,  'PER'],
    ['2', 259742,  'LOC'],
    ['1', 269340,  'PER'],
    ['2', 269340,  'LOC']
]

task = nltk.metrics.agreement.AnnotationTask(data=toy_data)

print('kappa', task.kappa() )
print('alpha', task.alpha() )
print('average Agreement', task.avg_Ao() )
print( 'pi', task.pi() )
print( 's', task.S() )

toy1 = ['ORG','LOC','PER','PER']
toy2 = ['ORG','LOC','LOC','LOC']
cm = nltk.metrics.ConfusionMatrix(toy1,toy2)
print( cm )
```

More details about data collection, corpora development, and web scraping can be found from the following sources: Web Scraping with Python by Ryan Mitchell, Calix and Knapp (2011), and Natural Language Processing with Python by Bird, Klein, and Loper.

3.6 Summary

In this chapter, a discussion about issues related to data and data pre-processing was provided. In particular, the following topics were addressed: data pre-processing, reading data from csv files, types of features, corpora processing, and one-hot encoding. In the next chapter, we will begin our discussion of how to program out first deep learning models with PyTorch.

4 Deep Learning: Starting at the beginning

In this chapter, I will introduce the topic of deep learning. My goal here is to help students and practitioners alike to get started with deep learning. In particular, I want them to be able to get data and pre-process it for use in deep learning, load it on to their models for training, perform classification and regression tasks, and evaluate their results. In this chapter, I will discuss some aspects of the theory providing as much intuition as possible. For this task, I will use Python, Sklearn, Pandas, the PyTorch framework, etc. I will also discuss some aspects of using GPU based hardware by the end of the book. For most of my sample code, I used the conda environment. I installed the PyTorch framework following the instructions in the main PyTorch website (https://www.PyTorch.org/).

4.1 Things to know about the code

As in the previous sections, I have included the libraries first. Notice, that I use as many **sklearn** libraries as possible and only use the PyTorch module when absolutely necessary. The goal, again, is to help the reader to best capture the PyTorch concepts by simplification. Once your skills improve with deep learning frameworks, you will be able to more easily replace sklearn modules with alternative PyTorch modules.

Listing 4.1: Libraries

```
import torch
import numpy as np
from numpy import genfromtxt

from sklearn import datasets
from sklearn.datasets import fetch_mldata
from sklearn.cross_validation import train_test_split

import sklearn
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score, f1_score
import pandas as pd
import matplotlib.pyplot as plt
```

In the previous code section, it can be seen that for our deep learning code to work, we only need a few new libraries. The most important new library is Torch. This of course is the main source of our new code. We will still rely on **Numpy** to efficiently get the data although we can also get data with the statement:

sklearn.datasets import fetch_mldata

The next important section of our code is the section to set some parameters. Deep learning algorithms are iterative in the sense that they load samples in batches to avoid running out of memory. This is very important as it allows you to load millions of data samples in subsets called batches. Therefore, we can set up the parameters for batch processing and other tasks. In the code segment below, we can see that we want to load batches of 100 samples each. For a training set of 1,000 samples, we would divide the set into 10 bins of 100 samples each and therefore run 10 batches. Two additional parameters in this code segment are the **learning_rate** and the **n_epochs**. The **n_epochs** parameter represents the number of times that we will run our main **"for"** loop. This is the loop that we will run to provide data to our algorithms for training and testing. At this point, an optimization begins to occur which helps the supervised learning algorithm to learn from the samples. The **learning_rate** value (in this case 0.01) is a very important parameter in the learning algorithm. Simply speaking, it represents the step that is taken in a gradient descent algorithm to find an optimal solution. Think of this like a giant walking over a mountainous region with many peaks and valleys. He is trying to find an optimal location; if the step is too big, the giant could go from peak to peak and skip a valley altogether. On the other hand, if the step is to small the giant may take too long to move in or out of a valley. In terms of PyTorch, the learning rate can affect convergence.

Convergence is the term that means that the iterative algorithm is starting to tend to the optimal solution. The **learning_rate** can also be assigned a function instead of just a value. This helps to adjust the learning rate in real time while the algorithm is running. Some researchers have suggested that this type of optimization can provide better results than just using a fixed value learning rate. This is achieved using schedulers.

Listing 4.2: Parameters

```
learning_rate = 0.01
n_epochs      = 27000          #1000
batch_size    = 100
```

Deep learning algorithms in PyTorch consist of a basic structure. Simply put, they consist of:

- Preparing the dataset or DataLoaders
- Defining the NN architectures (Inference Function)
- Defining the loss function and optimizer
- Training the model and adjusting the weights

In the next section I will proceed to further elaborate on this.

The figure below presents the accuracy plot per epoch of a deep neural network as it is being trained. The **x** axis represents the epochs over time and the **y** axis represents the accuracy score. As can be seen, the accuracy is low at first but it starts to improve after a certain number of epochs when the classifier begins to learn and converge.

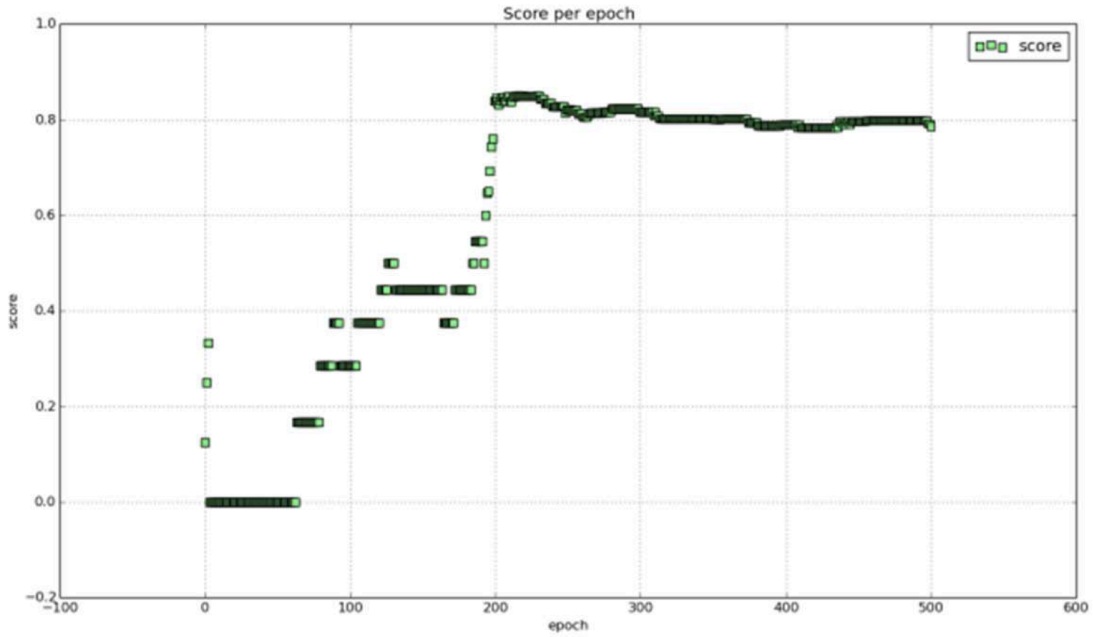


Figure 4.1: Accuracy per epoch

In this example on the figure, a deep neural network takes a few epochs to learn how to detect and classify the samples. After about 200 epochs, a deep net of say 2 hidden layers stabilizes and is able to maintain a consistent accuracy score.

4.2 Getting Started with Deep Leaning

So what's it all about? Again, my focus here is not to discuss deep learning from the equations point of view or talk about the derivatives. Those are all very important concepts but can be overwhelming when starting. Instead, I want to help the reader write deep learning code with PyTorch. Then, the reader will, no doubt, have many questions where the theory and fundamentals will help him or her to better understand and use the DL algorithms. The figures below present some of the challenges that must be addressed when dealing with real data. Real data such as language data from Twitter is highly noisy and un-balanced. An imbalance in the data means, for example, that 80 % of the samples belong to 1 class and the remaining 20 % belong to the other class. Therefore, training a classifier to, say, perform

emotion classification (where data is highly imbalanced) can be a difficult task. In the figure below, a sample of Twitter data is represented. Each sample in the dataset had multiple features but was compressed into a two dimensional vector for visualization purposes. As can be seen, the data in the figures has a high overlap in the classes and the data is difficult to separate. A linear classifier (the line seen on the graph) can only separate a small portion of the samples and the majority are not easy to separate in this space. The graph below is available on-line in color.

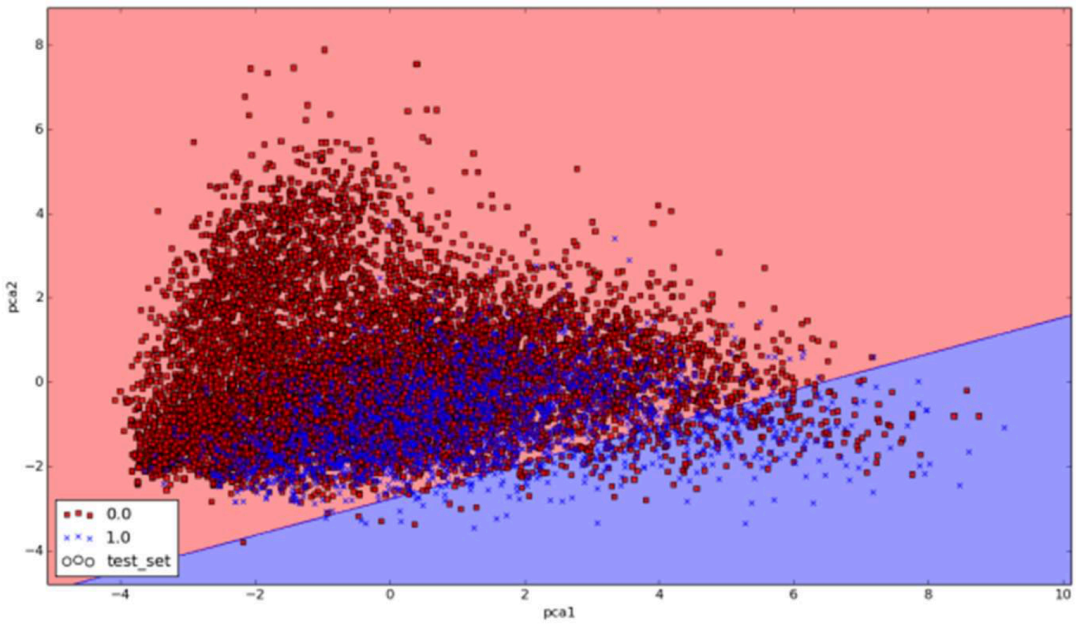


Figure 4.2: Logistic Regression Classifier

So how can we get more of the samples of one class without getting too many of the samples of the other class? Well, the answer is that we could use another type of line for the separation. For instance, instead of using a straight line we could use a curved line. This is why some algorithms are called linear and others are called non-linear. Non-linear algorithms can sometimes create better separations in the data and therefore obtain fewer errors. In the next Figure, an example of this is shown using Support Vector Machines (SVM). SVM methods can sometimes build better non-linear classifiers because of their ability to project data to higher dimensional spaces. Deep neural networks can be used to learn non-linear models as well. This property of non-linear models can help to obtain better classification accuracy scores.

This figure is available on-line in color.

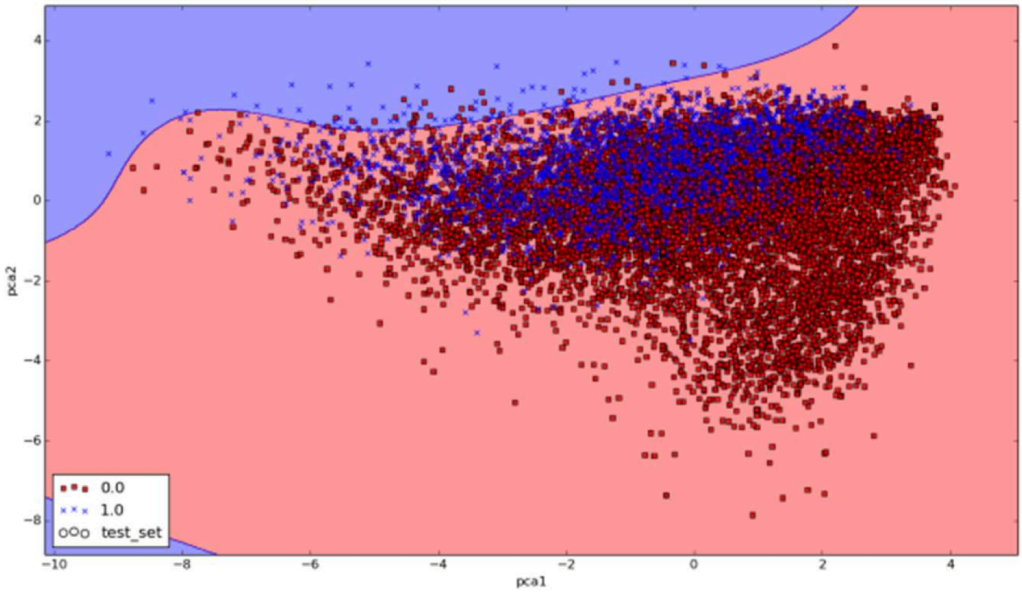


Figure 4.3: SVM Classifier

The figure above shows an SVM classifier building a non-linear separation line (the curved line in the graph) on the data. It could be expected that a deep neural network could build even better separation boundaries and that different architectures could give different results. This takes us to the very important aspect of deep learning architecture.

Deep learning architecture is where we define the parameters such as layers and neurons that define a deep neural network. You could think of this as the way that you construct the separation line that you want to build and use. We will discuss all of this in the next sections but it is important to know that you can define many deep learning architectures.

4.3 Deep Leaning Definition

Deep learning models are neural networks with many layers. As such, the more layers they have the deeper they are considered to be. In the next sections, I will begin to discuss neural networks. I will focus mostly on intuition when talking about neural networks. I will use linear regression and logistic regression constructs to define them first. We can also think of deep neural nets as functions and this will become very useful as we start to develop our code. Additional discussions of deep learning theory can be found in many other books such as "Fundamentals of Deep Learning: Designing Next-Generation Machine Intelligence" by Nikhil Buduma and Nicholas Locascio, and Deep Learning by Goodfellow et al. (2016).

4.4 PyTorch Basics

The main idea with the implementation of deep neural networks in PyTorch is that you need to define a computational graph and run your code through this graph on the CPU (s) or GPU (s).

Here are some of the main aspects of PyTorch to be aware of:

- PyTorch is object oriented
- Tensors are multidimensional arrays
- PyTorch variables are memory buffers that contain these tensors
- We call elements from PyTorch by referencing the object "torch" as such: **torch.nn**
- In PyTorch you need to define a graph and then run it
- The gradients (derivatives) are calculated using PyTorch by invoking a few simple functions

It is now time to write some PyTorch code. Here, let us add 2 numbers with PyTorch.

Listing 4.3: Adding 2 numbers with PyTorch

```
a = torch.tensor( [10] )
b = torch.tensor( [32] )

def model(a, b):

    return a + b

res = model(a, b)
```

In the previous code segment we initialize 2 variables **a** and **b** as torch tensors, and add them together. What is important to note here is that whenever you reference an object in PyTorch with **torch**, such as initializing the variable **a**, you are actually adding elements or nodes to a graph. The graph can be seen in the following figure.

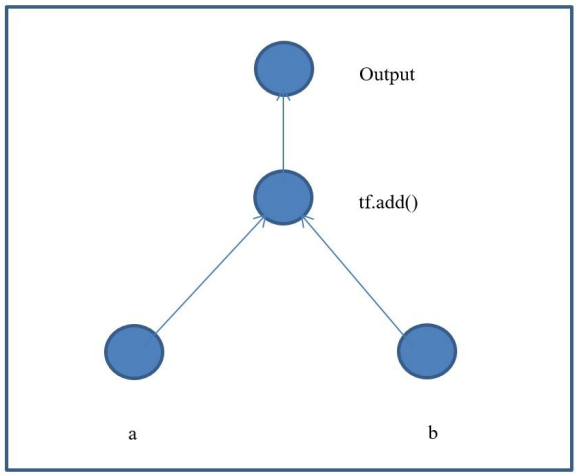


Figure 4.4: PyTorch graph

In this case, the command adds the 2 numbers together. The answer for this problem, after printing the results, should be 42.

4.5 Loading your Data into your PyTorch code

Loading the data can actually be somewhat complicated in PyTorch. One of the issues is that you may need to convert the data, say from something like the below example, into another format using one-hot encoding for the labels, for instance.

Listing 4.4: Iris data

```
class,f1,f2,f3,f4
1.0,6.1,2.8,4.7,1.2
0.0,5.7,3.8,1.7,0.3
2.0,7.7,2.6,6.9,2.3
1.0,6.0,2.9,4.5,1.5
1.0,6.8,2.8,4.8,1.4
0.0,5.4,3.4,1.5,0.4
1.0,5.6,2.9,3.6,1.3
2.0,6.9,3.1,5.1,2.3
1.0,6.2,2.2,4.5,1.5
1.0,5.8,2.7,3.9,1.2
```

Here, we are looking at the Iris data set. The first column represents the annotated label. The Iris dataset has 3 classes (0, 1, and 2). The other 4 columns represent the features per each sample. Each row is a sample. The code provided below will show how to read in the data. In the code below, we assume that the data is contained in the file iris.csv (available at the book github). We can use the Numpy **loadtxt()** function to obtain the data and to load it into a Numpy matrix. Notice that the file has a header for the class column and the 4 feature columns. We can easily remove the header by using the parameter **skiprows=1** in the **numpy.loadtxt()** function.

Listing 4.5: Iris data slicing

```
f_numpy = open("data/iris.csv",'r')

Matrix_data = numpy.loadtxt(f_numpy, delimiter=",", skiprows=1)

#X=Matrix_data[:, [1,2,3,5,6]]

X = Matrix_data[:, 1:4]
y = Matrix_data[:,  0]
```

In the previous code listing we can see code statements to slice the class column into the **y** vector and the rest of the columns into the **X** matrix. Notice that we are following the SKlearn approach from the previous chapters. PyTorch has its own approaches but I have used SKlearn here for the sake of simplicity. When slicing a 2-D matrix (**matrix[i, j]**) we specify the row index with **"i"** and the column index with **"j"**. We can also specify ranges by doing the following:

matrix[2:5, :]

The **":"** on the **"j"** index indicates that we should select all columns. If we wanted to select from a list of column indices we could do the following:

X=Matrix_data[:, [1,2,3,5,6]]

The next code listing shows us how to split the data.

Listing 4.6: Train test split

```
X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.20,
            random_state=42
)
```

Similarly to the previous chapters, we can now split the data from **X** and **y** to obtain the train and test sets: **X_train**, **X_test**, **y_train**, **y_test**. As can be seen in the previous code segment, these data sets are now ready for one-hot encoding (if needed). Only the labels need to be one-hot encoded. The code for performing the one hot-encoding step is below. The one-hot encoding function and process were defined in the previous chapter. As of 2024, PyTorch is so helpful that it can automatically do the one-hot encoding for you.

However, for completion let us see how to do this manually.

Listing 4.7: One hot function in use

```
y_train_onehot = convertOneHot_data2(y_train)
y_test_onehot  = convertOneHot_data2(y_test)
```

In the code above, we take the labels in **y_train** and **y_test** and pass them through our one-hot encoding function to produce the new variables **y_train_onehot** and **y_test_onehot**. You can visualize the new one-hot encoded vectors by running the following:

print(y_train_onehot[:20,:])

The print statement shows the first 20 samples. We will see this in later chapters.

Listing 4.8: The one-hot encoding function

```
# Convert to one hot data
def convertOneHot_data2(data):
    y=np.array([int(i) for i in data])
    #print y[:20]
    rows = len(y)
    columns = y.max() + 1
    a = np.zeros(shape=(rows,columns))
    #print a[:20,:]
    print rows
    print columns
    #rr = raw_input()
    #y_onehot=[0]*len(y)

    for i,j in enumerate(y):
        #y_onehot[i]=np.array([0]*(y.max() + 1) )
        #y_onehot[i][j]=1
        a[i][j]=1

    return (a)
```

The next step in the data processing is to scale the **X** data. I suggest using your data with scaling and without to compare the performance of your model. Convergence of the deep learning algorithm and the model's results can be better with scaled data. In this case we can use SKlearn's scaling module **StandardScaler()** to perform this task.

Listing 4.9: Scaling

```
sc = StandardScaler()

sc.fit(X_train)

X_train_normalized = sc.transform(X_train)
X_test_normalized  = sc.transform(X_test)
```

As can be seen in the above code segment, we only need to scale the **X** data. The **y** vector can be scaled as well, if needed. We can fit a scaling model with the train set and then use that model to scale both the **X_train** and **X_test** sets.

Now that our data is ready, we may want to save some information about the dimensions of the data sets in a couple of variables (**"A"** and **"B"**). This can be very helpful for code re-usability. We will store the number of features in **"A"** and the number of classes in **"B"**.

To obtain the dimension values, we can use the **.shape[z]** function available to Numpy arrays. In the code below, we can see an example of how to get this value for **"A"**. In this case, **"z"** represents the matrix dimension (number 0 for rows, and number 1 for columns).

Listing 4.10: Print feature and class parameters

```
# features (A) and classes (B)
# A = number of features
# B = number of classes
# 10 numbers for Mnist (0,1,2,3,4,5,6,7,8,9)
# 4 numbers for iris


A = X_train_normalized.shape[1]
#num features

B = y_train_onehot.shape[1]
#num classes

samples_in_train = X_train_normalized.shape[0]
samples_in_test  = X_test_normalized.shape[0]

print("num features", A)
print("num classes",  B)
print("num samples train", samples_in_train )

print("num samples test", samples_in_test)
```

At this point, we have completed most of the more common data pre-processing steps and we are ready to define the DL algorithms. This is where we define the neural network architecture, the cost functions to be used, and the functions to predict results for given test samples. In the next section, I will implement a linear regression algorithm in PyTorch. After that, I will extend the linear regression framework to build a logistic regression model. Finally, we will implement DL architectures.

4.6 Linear Regression

Linear regression refers to a model used for predicting any kind of magnitude (such as housing prices, sales prices, ages, temperatures, etc.) instead of just classes (e.g. happy vs. sad, etc.). In the next subsections I will cover the theory, intuition, and implementation of linear regression with PyTorch.

4.6.1 Theory and Intuition of Linear Regression

Two important concepts (functions) to consider when developing any NN are: 1) the inference equation and 2) the loss function.

Essentially, all NNs have these 2 functions.

- inference equation
- loss function

In linear regression, multiple variables and coefficients are combined to form an equation (inference function) that can be used to fit a particular data set. The fitting process involves an optimization approach that minimizes a loss function called MSE (Mean Squared Error).

The coefficients or weights for each variable in the inference function are useful in determining the contribution of each variable to the overall fitted model. The definition of the least squares error function can be seen in the equation below (also known as MSE).

$$MSE = \frac{1}{n} \sum_{i=0}^{n} (y_i - (w * x_i + b))^2$$

Here, **n** represents the number of samples in the dataset.

In linear regression, a line is fitted to the dataset in feature space by minimizing the sum of errors. That is to say, the sum of the error distances between the values in the predicted line and the real values are estimated (see figure below). Multiple lines can be fitted to the data. The goal is to find the line that minimizes the errors.

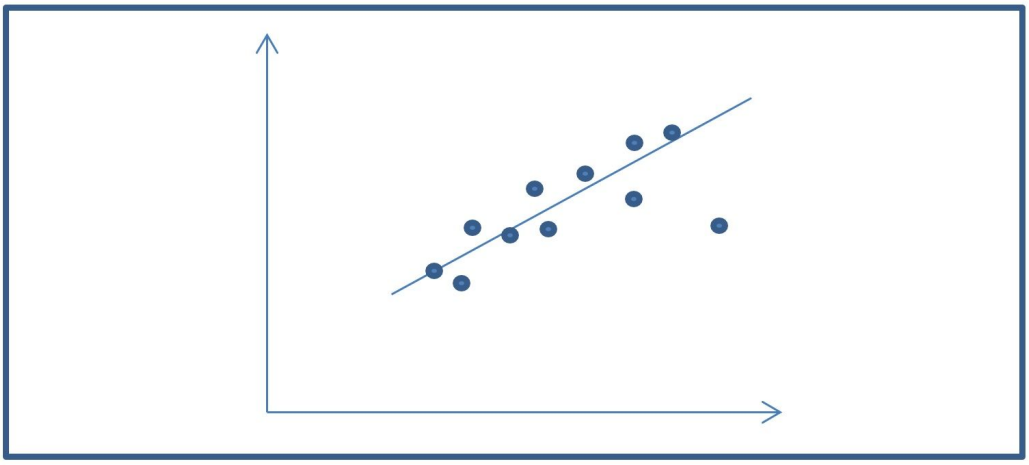


Figure 4.5: Linear Regression Line

A linear regression model can be defined by the following equation.

In the equation, the term $\left\langle \bullet, \bullet \right\rangle$ denotes the dot product in the input space between **w** and **x**. Here, **x** is the input vector, **w** is the weight vector, and **b** is the bias.

$$f(x) = \left\langle w, x \right\rangle + b$$

This simple equation defines the linear regression model and the parameters that must be learned given the training data.

Let us consider a simple example. We are going to assume a very simple regression problem for housing prices. We are going to have 1 input x_1 (the size of the house) and we will predict 1 output value y (the cost of the house). Therefore, we will have an equation as such:

$$y = w_1 * x_1 + b$$

For this case, we would need to solve this equation by learning the values of w_1 and b (the parameters) given the training data for x_1 and y.

Predicting the cost of the house just based on size may be insufficient. Instead, we could add a second feature (x_2). The equation would then be modified as follows:

$$y = w_1 * x_1 + w_2 * x_2 + b$$

We can show the previous equation in diagram form as follows:

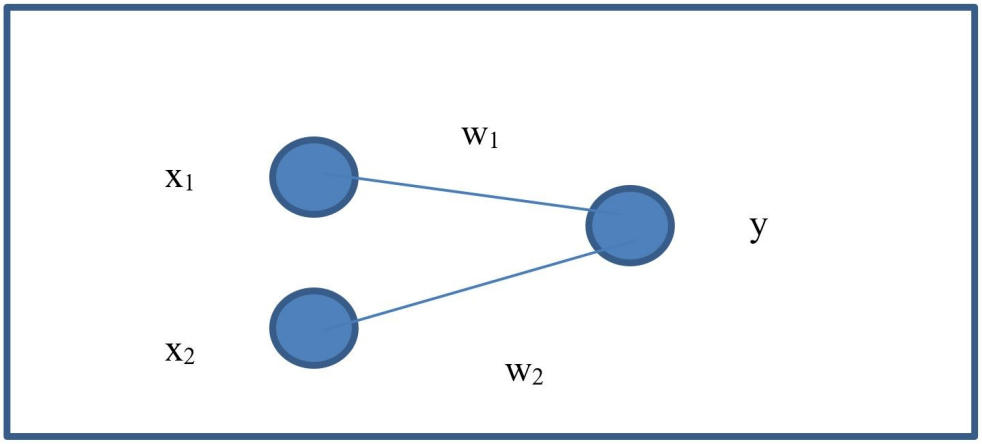


Figure 4.6: Linear Regression Architecture

As shown in the previous equation, a linear regression model can be represented as an equation made up of 3 main elements: **x**, **w**, and **b**. The **x** value holds the training data (features), **w** represents the weights per feature, and **b** is the offset value (bias) learned by the model. These values are directly coded into our algorithm using PyTorch.

The 2 variables, **w** and **b**, define the prediction equation. They are the parameters to be learned. All examples in this book will use these variables whether we are defining linear regression, logistic regression, or deep neural nets. **W** and **b** are matrices (tensors) whose dimensions need to be defined. This is where we start to define the architecture of our models. For this example, the dimensions are [2,1] for W and just [1] for b. In the case of W, the

dimension is [2, 1] because it represents a model with 2 features (x_1 and x_2) and 1 output value (the predicted housing price y). Think of **W** as a matrix that connects one layer to the next. In this case, we are connecting the inputs to the outputs.

In the next section we proceed to train a linear regression model for a wine quality dataset using PyTorch.

4.6.2 Linear Regression model for Wine Quality data

In this section a linear regression model is trained with PyTorch for a Wine Quality Dataset. Our Wine Quality dataset can also be used for classification since the **y** values are discrete and consist of discrete values as follows: [0, 1, 2, 3, ..., 10]. These values represent the wine quality rating from 0 to 10.

Therefore, this Wine Quality dataset can be used to illustrate the differences between regression and classification modeling.

Let us now start coding your first PyTorch model for regression.

We first add some of our common libraries.

Listing 4.11: Common Libraries

```
import numpy as np
import torch
import pandas as pd
import sklearn
import random
import matplotlib.pyplot as plt

from mlxtend.plotting import heatmap
from sklearn.model_selection import train_test_split
from torch.utils.data import TensorDataset, DataLoader
from sklearn.metrics import r2_score
```

The following code listing shows the main libraries used to define neural network architectures in PyTorch.

Listing 4.12: PyTorch NN specific libraries

```
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
```

We set the parameters in the next code listing. These include the learning rate, the number of epochs, and the batch size.

Listing 4.13: NN parameters

```
batch_size    = 16
learning_rate = 0.0003
N_epochs      = 200        ## 1000

epsilon = 0.0001
```

With the parameters set, we can now proceed to read in the data.

Listing 4.14: Read in the Wine data

```
path_data = 'winequality-white.csv'

WINE_raw_data_df = pd.read_csv(path_data, delimiter=";")
```

We can print the data as a Pandas data frame with the following statement:

Listing 4.15: print the wine data

```
print( WINE_raw_data_df )
```

The pandas data frame can be seen in the next figure:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
File display	7.0	0.27	0.36	20.7	0.045	45.0	170.0	1.00100	3.00	0.45	8.8	6
1	6.3	0.30	0.34	1.6	0.049	14.0	132.0	0.99400	3.30	0.49	9.5	6
2	8.1	0.28	0.40	6.9	0.050	30.0	97.0	0.99510	3.26	0.44	10.1	6
3	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.99560	3.19	0.40	9.9	6
4	7.2	0.23	0.32	8.5	0.058	47.0	186.0	0.99560	3.19	0.40	9.9	6
...	...	...	...	...	...	...	...	...	...	...	...	...
4893	6.2	0.21	0.29	1.6	0.039	24.0	92.0	0.99114	3.27	0.50	11.2	6
4894	6.6	0.32	0.36	8.0	0.047	57.0	168.0	0.99490	3.15	0.46	9.6	5
4895	6.5	0.24	0.19	1.2	0.041	30.0	111.0	0.99254	2.99	0.46	9.4	6
4896	5.5	0.29	0.30	1.1	0.022	20.0	110.0	0.98869	3.34	0.38	12.8	7
4897	6.0	0.21	0.38	0.8	0.020	22.0	98.0	0.98941	3.26	0.32	11.8	6

4898 rows × 12 columns

Figure 4.7: Wine data in pandas data frame

Now we can print the features in the Pandas data frame as follows:

Listing 4.16: The features in the data frame

```
headers_list = WINE_raw_data_df.columns.values.tolist()

print( headers_list )
```

Running the previous code listing gives us the following features:

Listing 4.17: Wine dataset features

```
['fixed acidity',
 'volatile acidity',
 'citric acid',
 'residual sugar',
 'chlorides',
 'free sulfur dioxide',
 'total sulfur dioxide',
 'density',
 'pH',
 'sulphates',
 'alcohol',
 'quality']
```

We can also select a subset of features to focus on as follows. Notice that in Pandas we can just use the column name.

Listing 4.18: A subset

```
headers_list2 = [ 'density',
 'pH',
 'sulphates',
 'alcohol',
 'quality']

print( headers_list2 )
```

Printing this subset should give us 5 features as shown below.

Listing 4.19: Printed selected features

```
['density', 'pH', 'sulphates', 'alcohol', 'quality']
```

To get a better understanding of the features, we can generate a correlation matrix.

We can do that with the following code.

Listing 4.20: Code to print correlation matrix

```
cm = np.corrcoef(    WINE_raw_data_df[ headers_list ].values.T   )
hm = heatmap(cm, row_names=headers_list , column_names=headers_list , figsize=(20,10))
```

Running the previous code gives us the following correlation matrix.

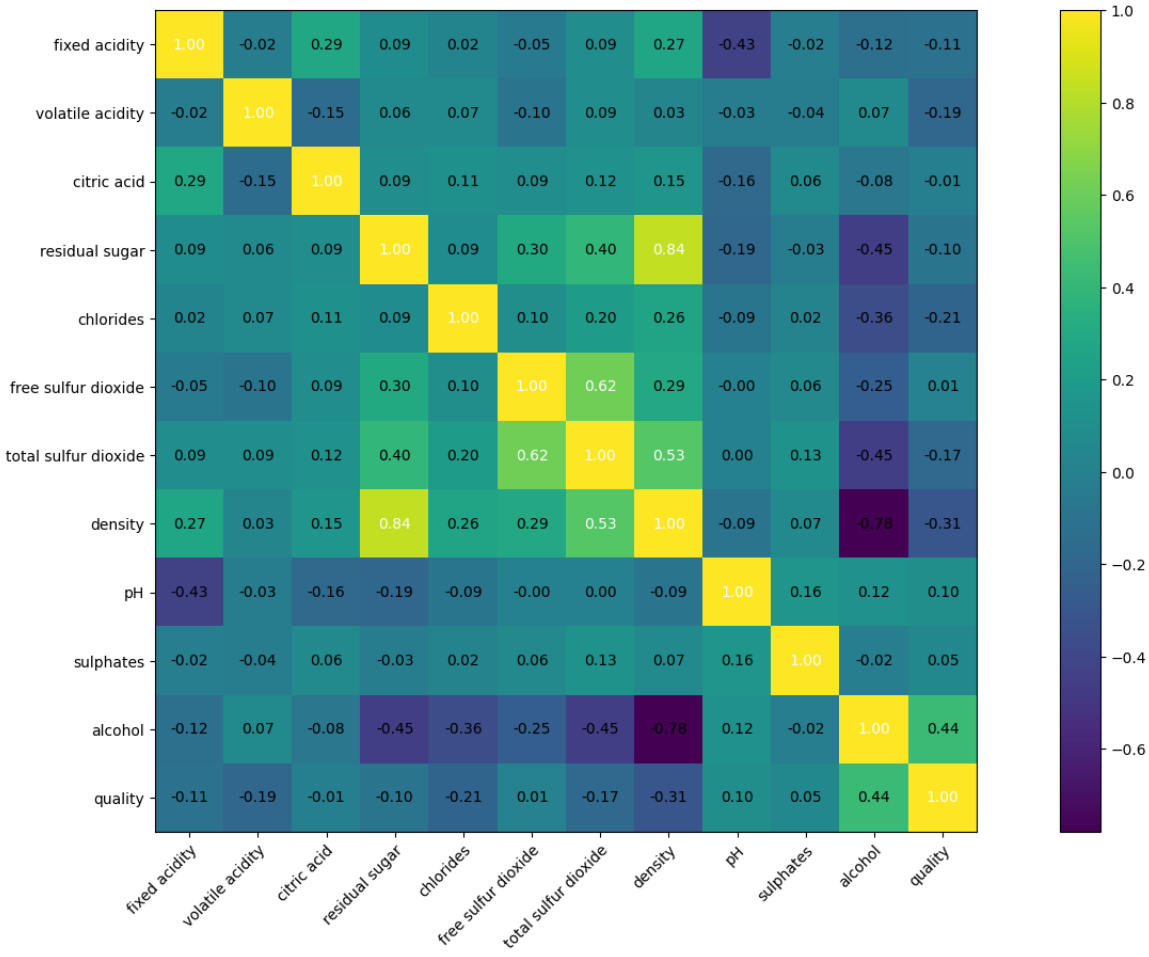


Figure 4.8: Wine data correlation matrix

The key observation with a correlation matrix is that intersecting rows and columns with values very close to zero have no correlation, whereas intersecting rows and columns with values close to one (either positive or negative) have strong correlation.

Now we can proceed to process the data and convert it from Pandas to Numpy.

Listing 4.21: Convert pandas wine data to numpy

```
WINE_raw_data_np = WINE_raw_data_df.to_numpy()

print( WINE_raw_data_np )
```

The previous code segment will also print the Numpy array as follows:

Listing 4.22: Wine data as numpy

```
array([[ 7.   ,  0.27,  0.36, ...,  0.45,  8.8 ,  6.  ],
       [ 6.3  ,  0.3 ,  0.34, ...,  0.49,  9.5 ,  6.  ],
       [ 8.1  ,  0.28,  0.4 , ...,  0.44, 10.1 ,  6.  ],
       ...,
       [ 6.5  ,  0.24,  0.19, ...,  0.46,  9.4 ,  6.  ],
       [ 5.5  ,  0.29,  0.3 , ...,  0.38, 12.8 ,  7.  ],
       [ 6.   ,  0.21,  0.38, ...,  0.32, 11.8 ,  6.  ]])
```

Knowing the shape of your tensors in DL is essential. If we print the shape for our tensor we can see it is (4898, 12). That means that we have 4898 samples with 11 features each, plus one class.

Listing 4.23: Print shape

```
print( WINE_raw_data_np.shape )
```

We can now proceed to slice the wine data into **X** and **y** which gives tensors with the following dimensions for **X** and **y**.

X = (4898, 11)

y = (4898, 1)

We can see that we have 11 features and one inference value for the class. The code to slice the data is shown in the next code listing.

Listing 4.24: slice wine data into x and y

```
X = WINE_raw_data_np[:, :-1]

y = WINE_raw_data_np[:, 11:12]

print( X.shape )

print( y.shape)
```

Remember here that the -1 index is used to get the last value in the sequence. Printing the **y** Numpy vector gives us the following:

Listing 4.25: The y numpy vector

```
array([[6.],
       [6.],
       [6.],
       ...,
       [6.],
       [7.],
       [6.]])
```

It is good practice to split the data randomly every time you train the model. By doing this, you get to see the performance variation across training attempts. The following implementation is what I usually call a ”poor person’s ten fold cross validation”. I can achieve it quite simply by choosing a new random seed every time I split the data.